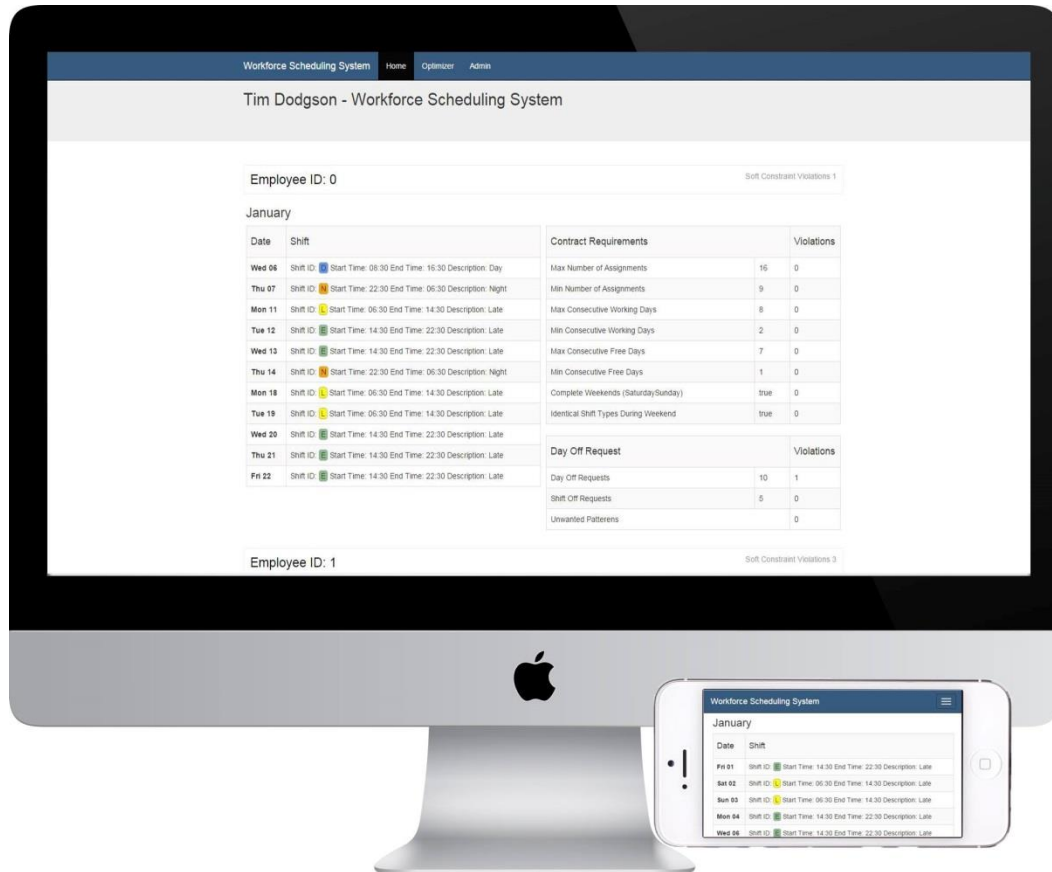


Tim Dodgson

BSc (Hons) Computing

Workforce Scheduling Web Application



Supervisor Dr Alan Crispin

Software Download:

https://www.dropbox.com/s/d7dpc8gh613tbd3/Tim_Dodgson_Project.rar?dl=0

Live version: <http://wfs.uk.to/Demo>

username level 1: 1 ... 10

password passdemo

username level 2: admin

passadmin

Disclaimer

No part of this project has been submitted in support of an application for any other degree or qualification at this or any other institute of learning. Apart from those parts of the project containing citations to the work of others, this project is my own unaided work.

Signed _____

Acknowledgements

Thank you to Dr Alan Crispin, his constant support throughout my final year project and other studies in general has been remarkable.

Thanks go to my friends and family, who have helped me over the last few years.

Abstract

The general workforce scheduling problem involves the assignment of employees to specific working days and shifts in order to cover the scheduling demands. The problem size is determined by the planning horizon, number of employees and flexibility of the shifts. Further complexity is added when employees have to be assigned tasks within a shift.

The scheduling optimizer constraints are those which must be true to achieve a feasible solution. An assignment which satisfies all hard constraints is known as a feasible has to consider both hard constraints and soft constraints. Hard constraints are those which are always valid while soft constraints are those we would like to be valid but are not critical to produce a feasible solution. The general goal is to schedule the resources to meet the hard constraints while aiming at a high quality result with regards to the soft constraints.

Table of Contents

Chapter 1 Introduction	1
Chapter 2 Literature Review	2
2.2 Workforce Scheduling Problems	2
2.3 Nurse Rostering Problem	2
2.3.1 Nurse Rostering Problem Data Sets	2
2.3.2 Location	3
2.4 Iterative Improvement Search	3
2.4.1 Iterative Improvement	3
2.4.2 Hill Climbing	3
2.4.3 Genetic Algorithm	4
2.4.4 Late Acceptance Hill Climbing	4
2.4.5 Iterated Local Search (Hill Climbing)	5
2.4.6 Simulated Annealing	5
2.4.7 Other Metaheuristics	6
2.4.8 Comparing Metaheuristics	6
2.5 Parsing Data Sets	7
2.5.1 Simple API for XML (SAX)	7
2.5.2 The Document Object Model (DOM)	7
2.5.3 Java Architecture for XML Binding (JAXB)	8
2.6 Web Application Development	8
2.6.1 Java Servlets	8
2.6.2 Extensible Markup Language (XML)	8
2.6.3 Extensible Stylesheet Language (XSL)	9
2.6.4 Presentation	9
2.7 Summary	9
Chapter 3 Schema Design for Staff Rostering	10
3.2 Data binding an schema XSD file	10
3.3 Employee rostering schema (XSD)	13
3.5 Extend schema to include location data and patients	16
3.6 Summary	17
Chapter 4 Design and Implementation of the Workforce Scheduling Software	18
4.1 Introduction	18
4.2 Design of the Workforce Scheduling Software	18

4.3 Implementation of the Workforce Scheduling Software	18
4.3.1 Initialise schedule with feasible solution	19
4.3.2 Soft Constraint Violations	21
4.3.2.1 Soft Constraints that are standard across all Employees	21
4.3.2.2 Soft Constraints that are specific to an Employees contract	21
4.3.2.3 Soft Constraint violation detection	22
4.3.3 Local Operators	23
4.3.4.1 Simulated Annealing	24
4.3.4.2 Iterated Local Search	25
4.4 Extending Nurse Rostering Schedule to Cluster patient visits to staff locations	25
4.4.1 Clustering Patients	26
4.4.2 Local Operators	26
4.5 Summary	27
Chapter 5 Evaluation of Workforce Scheduling	28
5.1 Evaluation of Metaheuristics for WFS	28
5.2 Evaluation of objectives Achieved	30
5.2.1 Literature review objectives	30
5.2.2 Design objectives	30
5.2.3 Implementation objectives	30
5.2.4 Initial advanced objectives	31
5.3 Usability Testing	31
5.3.1 Application tested using different browsers on different devices	31
5.3.2 Responsive web design (RWD)	34
5.3.3 Evaluation of Patient to Nurse Clustering	34
Chapter 6 Web Deployment and Configuration	36
6.1.1 Operating system selection	36
6.1.2 Configuring Virtual Private server	36
Access to the server is via the console, once logged in as root (Linux Super User) the first step is to add a new user and configure the new user as a sudo.	36
6.1.3 Tomcat Web Server Installation and Configuration	37
6.1.3 Nginx	38
6.1.4 Install and configure	39
6.1.5 Deploying Workforce Scheduling Web Application	40
6.2 Servlets	42

6.3 Transforming output	43
6.3.1 XSL	44
6.4 Bootstrap	45
6.5 Understanding the schedule	46
Chapter 7 Conclusions and Future Work	47
7.1 Meta-Heuristics	47
7.2 Local Operators	47
7.3 Virtual Private Server	47
7.4 Future work	47
Appendix	48

Table of Figures

Figure 1 Steps in the JAXB Binding Process [4].	10
Figure 2 Console JAXB test output.	15
Figure 3 Overall program flow.	18
Figure 4 Use Case Diagram.	19
Figure 5 UML Initialise Feasible Schedule.	20
Figure 6 Initialised Feasible Schedule.	20
Figure 7 UML Soft Constraint Violations.	22
Figure 8 Conversion of employee schedule to string.	22
Figure 9 Swapping two shifts with single random swap Local Operator.	23
Figure 10 UML Local Operator.	24
Figure 11 Tuning Simulated Annealing.	25
Figure 12 Tuning Iterated Local search.	25
Figure 13 Optermized Clustering of patients.	26
Figure 14 Simulated Annealing two minute pass.	28
Figure 15 Iterated Local Search two minute pass.	29
Figure 16 Internet Explorer.	32
Figure 17 Firefox.	32
Figure 18 Google Chrome.	33
Figure 19 iPhone.	33
Figure 20 Responsive website design.	34
Figure 21 Patient clustering.	35
Figure 22 Putty.	36
Figure 23 Tomcat it Works page.	37
Figure 24 Tomcat it configuration page.	38
Figure 25 Create WAR file in eclipse.	40
Figure 26 Contents of WAR file.	41
Figure 27 Deployed website.	42
Figure 28 XSLT Processor.	43
Figure 29 Responsive display.	45
Figure 30 Responsive display.	46
Figure 31 Roster Explained.	46

Table of Tables

Table 1 Comparison of Meta-heuristics.....	7
Table 2 Simulated Annealing comparison of different decent curves.....	30
Table 3 Iterated Local Search comparison of different termination condition and kick values.....	30
Table 4 Literature review objectives.....	30
Table 5 Design objectives.....	30
Table 6 Implementation objectives	31
Table 7 Advanced objectives	31

Abbreviations

XML	Extensible Markup Language
HTML	Hyper Text Markup Language
CSS	Cascading Style Sheets
XSD	XML Schema Definition
API	Application Programming Interface
MVC	Model View Controller
JS	JavaScript
XSL	Extensible Stylesheet Language
GUI	Graphical User Interface
UML	Unified Modeling Language
UCD	Use Case Diagram
SQL	Structured Query Language

Chapter 1 Introduction

The Workforce Scheduling Web Application will take a known definition data set that describes a typical hospitals staffing roster. The data set stores lists of available employees, skills needed, shift types and shift coverage. Hard and soft constraints are also described.

A number of meta-heuristics both trajectory and population based will be compared to find the optimal feasible schedule that satisfy all hard constraints and as many soft constraints as possible.

Object orientated design patterns will be used to develop a flexible Java application that will firstly construct a feasible schedule (a feasible schedule meets and satisfies all hard constraints) then applies meta-heuristics to optimize soft constraint violations.

An XML representation of the optimized schedule and a XSL transform are used to produce HTML output to render the schedule in a web page.

Meta-heuristics algorithms require a certain degree of tuning, each algorithm will be tested with a range of different parameters. Once the optimal two algorithms have been decided on a comparison will be taken.

The optimal schedule will also be extended to give the option to assign in-home patients to mobile-nurses for a given day with the emphasis on reducing the overall distance travelled.

Web Application is fully password protected with user level access control, a standard user has access to their own schedule for the next planning horizon, where as an administrator has access to all normal users planning horizons as well as the optimizer and parameter tuning page and lastly the administration page.

Design of the website will employ responsive web design resulting in less code development and a standard look and feel across all devices be it desktop, table or smart phone.

Chapter 2 Literature Review

2.1 Introduction

The general workforce scheduling problem involves the assignment of employees to specific working days and shifts in order to cover the scheduling demands. The problem size is determined by the planning horizon, number of employees and flexibility of the shifts. Further complexity is added when employees have to be assigned tasks within a shift.

The scheduling optimizer constraints are those which must be true to achieve a feasible solution. An assignment which satisfies all hard constraints is known as a feasible has to consider both hard constraints and soft constraints. Hard constraints are those which are always valid while soft constraints are those we would like to be valid but are not critical to produce a feasible solution. The general goal is to schedule the resources to meet the hard constraints while aiming at a high quality result with regards to the soft constraints.

2.2 Workforce Scheduling Problems

Employee scheduling problems arise in a variety of service delivery settings, including the scheduling of nurses in hospitals, check encoders in banks, airline and hotel reservation personnel, telephone operators, patrol officers and others. In their simplest form, these problems involve only the assignment of days/shift off, as in some of the less complex settings for the scheduling of nurses. A typical problem of this type requires the scheduler to give appropriate days off to each of a number of employees who work standard shifts with differing start times while assuring that the required number of employees are on duty throughout the day and week. Variations of this type were addressed in Brownell and Lowerre [W. S. Brownell and J. M. Lawerre] and Keith [E. G. Keith]. This work focuses on Nurse Scheduling (Rostering) and related problems.

2.3 Nurse Rostering Problem

The nurse rostering problem (NRP) is the process of finding a duty roster for nurses in a hospital for a given horizon considering a large number of hard and soft constraints. The hard constraints, which have to be met to produce a feasible schedule, are different for each hospital, some core hard constraints are only one shift per employee per day and all shifts must be allocated. To measure the quality of a schedule the number of soft constraint violations is used. These constraints include personnel policies (e.g. maximum number of consecutive working days), preferences and days-off. A bibliographic survey is provided by Cheang et al. [B. Cheang, H. Li, A. Lim, and B. Rodrigues] where a wide variety of formulations is presented due to hospital-specific requirements. Burke et al. later provided a more detailed survey of the research done in the field of nurse rostering as well as an overview of the approaches.

2.3.1 Nurse Rostering Problem Data Sets

For testing the nurse rostering problem competition data sets from [Nurse Rostering Competition]. The sprint data sets are based on ten employees, working a twenty eight day schedule, each weekday has six shifts allocated and four shifts on weekends. Weekends are configurable and can be one of the following:

Saturday and Sunday
Friday, Saturday and Sunday
Saturday, Sunday and Monday

Friday, Saturday, Sunday and Monday

To produce a feasible schedule each employee can have a maximum of one shift per day and all shifts must be allocated. Further to this the sprint data sets have a number of soft constraints listed below:-

Maximum and Minimum working days for the entire schedule

Maximum and Minimum consecutive working days

Maximum and Minimum consecutive days off

If working weekend, work all weekend days and have the same shift on these days

Allow for unwanted patterns such as late night early

Day and Shift off requests

The optimiser should meet as many of the of the above constraints to produce an optimised schedule

2.3.2 Location

The schedule produced by the Nurse Rostering Problem optimiser shows which day and shift each nurse is working in the current planning horizon. By extending these data sets to include home locations for each employee and adding a list of outpatients which will also include the patients home locations, it will be possible to extend this optimiser to group outpatient visits to each employee. This would involve a second pass to further optimise each day of the schedule to group local outpatients to local nurses

2.4 Iterative Improvement Search

There are many types of optimiser that can be used to achieve good results for the Nurse Rostering Problem, different optimisers may work better for different planning horizons. Below is a list of popular search algorithms, with a brief description and where possible pseudo code.

2.4.1 Iterative Improvement

The basis of any iterative improvement algorithm is

- (i) start at a random configuration;
- (ii) repeatedly consider various different moves;
- (iii) accept & reject moves based on an acceptance criteria;
- (iv) if you get stuck in a local optima reshuffle solution and restart.

Generally you define a set of local neighbourhood operators or moveset. In the Travelling Salesman Problem (TSP) a moveset could consist of 2-opt, 3-opt, N-opt swaps etc. In NRP a moveset could consist of shift swaps in a day swapping 1, 2, 3 etc. swap(1), swap(2), swap (3) Moveset design is critical to getting a good solution.

2.4.2 Hill Climbing

HC(Hill Climbing) is a optimization technique that falls into the category of local search. HC is an iterative algorithm that produces a solution from a random place on the planning horizon, then by changing the solution, for instance on the NRP(Nurse Rostering Problem) the change may be swapping two shifts on one day on the planning horizon this results in a new neighbour solution

which is then compared to the current best and if is not worse is excepted as the new best solution. This action is repeated until a stopping condition is met, often a set number of iterations or a certain level of solution has been met[Wikipedia(Hill Climbing)].

Hill Climbing Pseudo code.

```

MaxIterations
InitialSolution = New Solution()
Current = Solution(InitialSolution)
FOR (I < MaxIterations)
    Neighbor = Solution(Current)
    LocalOperator(Neighbor)
    IF (Cost(Neighbor) <= Cost(Current))
        Current = Neighbor
    ENDIF
    I=I+1
ENDFOR
Code 01: Hill climbing pseudo code.

```

2.4.3 Genetic Algorithm

Genetic Algorithms (GAs) is a met-heuristic search algorithm based on the evolutionary ideas of natural selection and genetics. As such they represent an intelligent exploitation of a random search used to solve optimization problems.

Genetic Algorithm Pseudo code.

```

Algorithm: GA( $n$ ,  $\chi$ ,  $\mu$ )
// Initialise generation 0:
k := 0;
Pk := a population of n randomly-generated individuals;
// Evaluate Pk:
Compute fitness(i) for each i  $\in$  Pk;
do
{ // Create generation k + 1:
// 1. Copy:
Select  $(1 - \chi) \times n$  members of Pk and insert into Pk+1;
// 2. Crossover:
Select  $\chi \times n$  members of Pk; pair them up; produce offspring; insert the offspring into Pk+1;
// 3. Mutate:
Select  $\mu \times n$  members of Pk+1; invert a randomly-selected bit in each;
// Evaluate Pk+1:
Compute fitness(i) for each i  $\in$  Pk;
// Increment:
k := k + 1;
}
while fitness of fittest individual in Pk is not high enough;
return the fittest individual from Pk;
Code 02: Genetic Algorithm pseudo code.

```

2.4.4 Late Acceptance Hill Climbing

Late Acceptance Hill Climbing is also very similar to normal hill climbing with the addition of a history of previous solutions that are not as good a quality as the known best solutions, this means the algorithm accepts a candidate solution if it is not worse than that solution, which was current several steps before as well as the current one [C S Nott].

Late Acceptance Hill Climbing Pseudocode.

```

MaxIterations
InitialSolution = New Solution()
FitnessArray
LengthFitnessArray

```

```

FOR (I < LengthFitnessArray)
    FitnessArray[I] = InitialSolution
ENDFOR
Current = Solution(InitialSolution)
FOR (I < MaxIterations)
    Neighbor = RandomNeighbor(Current)
    v = I % LengthFitnessArray
    LocalOperator(Neighbor)
    IF (Cost(Neighbor) <= Cost(Current) || Cost(Neighbor) <= FitnessArray[v])
        Current = Neighbor
    ENDIF
    FitnessArray[v] = Cost(Current)
    I=I+1
ENDFOR

```

Code 03: Late Acceptance Hill Climbing pseudo code.

2.4.5 Iterated Local Search (Hill Climbing)

Iterated Local Search (ILS) is one of the most general scheme among the explorative strategies. On the one hand, its generality makes it a framework for other meta-heuristics (such as VNS(Variable Neighbourhood Search)), on the other hand other meta-heuristics can be easily incorporated as sub-components. ILS is a simple but powerful meta-heuristic algorithm [H. R. Lourenc,o, O. Martin, and T. Stutzle, H. R. Lourenc,o, O. Martin, and T. Stutzle, O. Martin, S. W. Otto, and E. W. Felten, T. Stutzle, T. Stutzle]. It applies local search to an initial solution until it finds a local optimum, then it perturbs the solution and it restarts a local search. Getting perturbation in an acceptable range is necessary, too small a perturbation might not enable the system to escape from the basin of attraction of the local optimum just found. On the other side, too strong a perturbation would make the algorithm similar to a random restart local search.

Iterated Local search Hill Climbing Pseudo code.

```

MaxIterations
MaxNoImprovement
InitialSolution = New Solution()
Current = Solution(InitialSolution)
FOR (I < MaxIterations)
    Neighbor = Solution(Current)
    IF (MNICount <= MaxNoImprovement)
        LocalOperator(Neighbor)
    ELSE
        Perturbation(Neighbor)
        MNICount = 0
    ENDIF
    IF (Cost(Neighbor) <= Cost(Current))
        Current = Neighbor
        MNICount = 0
    ELSE
        MNICount = MNICount + 1
    ENDIF
    I=I+1
ENDFOR

```

Code 04: Iterated Local search pseudo code.

2.4.6 Simulated Annealing

Simulated Annealing (SA) is a meta-heuristics algorithm that has an explicit strategy to avoid local minima. The origins of the algorithm are in statistical mechanics (Metropolis algorithm) and it was first presented as a search algorithm for CO (Combinatorial Optimization) problems in [V. Cerny] and [S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi]. The fundamental idea is to allow moves resulting in solutions of worse quality than the current solution (uphill moves) in order to escape from local minima. The probability of making such a move is decreased as the temperature is cooled.

Simulated Annealing Pseudocode.

```
Temp
CoolingRate
InitialSolution = New Solution()
Current = Solution(InitialSolution)
Best = Solution(Current)
FOR (Temp > 0)
    Neighbor = RandomNeighbor(Current)
    LocalOperator(Neighbor)
    IF (Cost(Neighbor) < Cost(Current))
        Current = Neighbor
        Best = Current
    ELSE (e (Cost(Current) - Cost(Neighbor) / Temp)
        Current = Neighbor
    ENDIF
    Temp *= CoolingRate
ENDFOR
```

Code 05: Simulated Annealing pseudo code.

2.4.7 Other Metaheuristics

Metaheuristics fall in two categories, trajectory based and population based. The biggest difference between these the two methods mentioned is the amount of solutions that is required in each step of the iterative algorithm.

Trajectory based algorithms start with a single solution and, at each step of the iterative search a neighbour solution is compared to the known best, if the neighbour solution is not worse it becomes the best solution. Trajectory-based metaheuristics allow to quickly find a locally optimal solution, and so they are called exploitation-oriented methods promoting intensification in the search space

Population based algorithms start with a population of solutions, the population is randomly generated and then enhanced through an iterated process. At each generation of the process, the whole population (or a part of it) is replaced by newly generated individuals (often the best ones). These techniques are called exploration-oriented methods, since their main ability resides in the diversification in the search space

An example of a population based algorithm is GA(Genetic Algorithm), due to large amount of tuning parameters to tune this and other population based algorithms the focus of this paper will be on trajectory based algorithms.

2.4.8 Comparing Metaheuristics

In the design and assessment of meta-heuristics the Berlin 52 travelling salesman instance was used for testing purposes as this has a known best answer (7542). Berlin 52 has 52 coordinates for locations around Berlin, by applying different algorithms to these coordinates the shortest route can be found. The literature review showed that the main four algorithms are Simulated Annealing, Iterated Local Search, Late Acceptance Hill Climbing and Genetic Algorithm. The Workforce Scheduling Web Application will use two algorithms for comparison so the Berlin 52 traveling salesman problem will be used as a test bed between the afore mentioned algorithms and the best two being used. Each algorithm will have one thousand passes and best and average vales will be compared. The table below shows the results.

	Best Result	Mean Average
Simulated Annealing	7542 (Known Best)	7929
Iterated Local Search	7542 (Known Best)	8091
Late Acceptance Hill Climbing	7542 (Known Best)	8311
Genetic Algorithm	7846	9119

Table 1 Comparison of Meta-heuristics.

The results of the Berlin 52 test clearly show Simulated Annealing and Iterated Local Search offer the best results, both hit the known best answer and have the least deviation on the mean average score

2.5 Parsing Data Sets

How to choose a parser:

DOM parser when:

- You need to know a lot about the structure of a document
- You need to change the structure of a document (maybe you want to sort elements, add some new ones, and so forth)
- You need to refer to the parsed information more than once

SAX parser when:

- You don't have much memory (your machine doesn't have much memory, that is)
- You only need a few elements or attributes from the XML document
- You'll only use the parsed information once

JAXB Java Architecture for XML Binding

JAXB is not a direct comparison to DOM and SAX. The Java DOM and SAX parsing APIs are lower-level APIs to parse XML documents, while JAXB (Java API for XML Binding) is a higher-level API for converting XML elements and attributes to a Java object hierarchy (and vice versa). Implementations of JAXB will most likely use a DOM or SAX parser behind the scenes to do the actual parsing of the XML input data. Often you'll want to convert the content of an XML document into objects in your Java program. If this is what you want to do, then JAXB will probably be easier to use and you'll need to write less code than when you would be using the DOM or SAX parsing API.[StackOverflow]

2.5.1 Simple API for XML (SAX)

Simple API for XML (SAX) is an application program interface (API) that allows a programmer to utilise data from a source that uses the Extensible Markup Language (XML). SAX is an alternative to using the Document Object Model (DOM) to interpret the XML file. As its name suggests, it's a simpler interface than DOM and is appropriate where many or very large files are to be processed, but it contains fewer capabilities for manipulating the data content.

SAX is an event-driven interface. The programmer listens for an event that may happen and, if it does, SAX gets control and handles the situation. SAX works directly with an XML parser

2.5.2 The Document Object Model (DOM)

A common interface for XML structures The DOM is an interface for working with the structure of

XML documents. A project of the W3C, the DOM was designed to provide a set of objects and methods to make life simpler for programmers. (Technically, the DOM predates XML; the early DOM work was meant for HTML documents.) Ideally, you should be able to write a program that uses one DOM-compliant parser to process an XML document, and then switch to another DOM-compliant parser without changing your code. When you parse an XML document with a DOM parser, you get a hierarchical data structure (a DOM tree) that represents everything the parser found in the XML document. You can then use functions of the DOM to manipulate the tree. You can search for things in the tree, move branches around, add new branches, or delete parts of the tree.

2.5.3 Java Architecture for XML Binding (JAXB)

JAXB allows Java developers to bind Java classes to XML instances. JAXB provides two main features: the ability to marshal Java objects into XML and unmarshal XML back into Java objects. In other words, JAXB allows storing and retrieving data in memory in any XML format, without the need to implement a specific set of XML loading and saving routines for the program's class structure. It is similar to xsd.exe and XmlSerializer in the .NET Framework.

JAXB is particularly useful when the specification is complex and changing. In such a case, regularly changing the XML Schema definitions to keep them synchronised with the Java definitions can be time consuming and error-prone.

JAXB is a part of the Java SE platform and one of the APIs in the Java EE platform, and is part of the Java Web Services Development Pack (JWS DP).

2.6 Web Application Development

The web application will be developed in java servlets and served via an Apache Tomcat server, this will give a vast amount of flexibility in development and offer flexibly in scaling the application. Servlets also offer the full strength and security of the java object oriented programming language which is a great concern in developing a cross platform application [Peter Lin]. A main servlet will run in a background thread, this will be responsible for running the optimiser, as better solutions are generated by the optimiser, they are passed to the front end servlet, who's main task is to produce XML and pass it to a XSL which will transform the XML into Hypertext Markup Language (HTML)

2.6.1 Java Servlets

The servlet is a Java programming language class used to extend the capabilities of a server. Although servlets can respond to any types of requests, they are commonly used to extend the applications hosted by web servers, so they can be thought of as Java applets that run on servers instead of in web browsers. These kinds of servlets are the Java counterpart to other dynamic Web content technologies such as PHP and ASP.NET.[Wikipedia (Java Servlet)]

2.6.2 Extensible Markup Language (XML)

Extensible Markup Language (XML) is a markup language that defines a set of rules for encoding documents in a format which is both human-readable and machine-readable. It is defined by the W3C's XML 1.0 Specification and by several other related specifications,[Sachidanand V. Begur, David M. Miller, and Jerry R. Weaver] all of which are free open standards.

The design goals of XML emphasize simplicity, generality and usability across the Internet.[5] It is a textual data format with strong support via Unicode for different human languages. Although the design of XML focuses on documents, it is widely used for the representation of arbitrary data

structures such as those used in web services.[Wikipedia (XML)]

2.6.3 Extensible Stylesheet Language (XSL)

XSL is a recursive programming language that is used to transform XML documents into different format. This offers great flexibility in separation of business logic and presentation of HTML [Wikipedia (XSL)]

2.6.4 Presentation

Presentation of the HTML is styled via CSS and can be further manipulated via JavaScript, due to differences between browsers and the device types that the web application will be viewed on, a number of frameworks have been developed that aid in the creation of responsive website a list of some more common frameworks follow:-

Foundation CSS Framework (160MB)
Twitter Bootstrap (83MB)
Gumby Framework
YAMC Framework

Due to the small footprint of twitter bootstrap and its wide acceptance in the online programming community as well as being backed by Twitter, Bootstrap will be used to handle the presentation of a responsive web site which maintains a standard theme across desktop/mobile and tablet.

2.7 Summary

In summary the focus of this project will be on the Nurse Rostering Problem (NRP). The NRP is a large and varied problem which can be approached from different angles. The sprint datasets available from [Nurse Rostering Competition] will be used as a benchmark. The datasets being used set out a scenario where a hospital has ten nurses who are scheduled over a twenty eight day period, each weekday six shifts must be fulfilled and at weekends four shifts must be covered. These are minimum requirements for a feasible schedule. Next by use of optimisation algorithms an attempt to meet further constraints such as day off requests will be made. The standard in the industry optimisers appears to be Simulated Annealing (SA). Simulated Annealing requires that the two parameters initial temperature and cooling rate have to be tuned for the algorithm to perform optimum performance. Hill Climbing (HC) in comparison has no tuning parameters which makes this an ideal candidate to compare to SA. HC can get caught in a local optima which results in a sub-optimal solution. This can be alleviated by using variants of hill climbing that have mechanisms to escape a local optima.

With more and more software services being delivered over the internet the user interface will be via a web browser. The application will be hosted on an Apache Tomcat server and Java servlets used for backend services and client interactions. The administration and optimisation pages will be password protected with an administration password and each client(nurse) page will have individual passwords and will also be viewable via the administration password. All the services will be delivered to the internet from Apache Tomcat via Nginx which will offer some security protection as well as offer user friendly URLs. The whole site will be delivered over https and Twitter Bootstrap will be used to deliver a responsive website which will maintain a standard layout and theme across desktop/mobile and tablet.

Chapter 3 Schema Design for Staff Rostering

3.1 Introduction

This chapter will explain the process of using JAXB to bind a definition XML document which contains a dataset that will be used in generating a feasible schedule. The XSD schema will be explained and some snippets of said schema displayed to further understand how the schema is used in the binding process. Once JAXB has read the definition XML document via the schema a set of Java classes will be generated with values prepopulated from the definition XML document ready for use in Java code. A short test script will be written to demonstrate connectivity to the definition XML document, this will be rendered at the console.

3.2 Data binding an schema XSD file

JAXB offers an easy and reliable method to model a set of Java classes that store values from a definition XML source; the class structure also reflects the hierarchical structure of the input XML. JAXB can be used in both directions so values stored in Java classes can be marshalled to an XML file or values for an XML document can be unmarshalled to Java classes, as this is a binding any changes to vales stored in an XML document are instantly reflected in the Java classes.

The Java Architecture for XML Binding () provides a powerful and practical way to work with XML content from within Java applications.

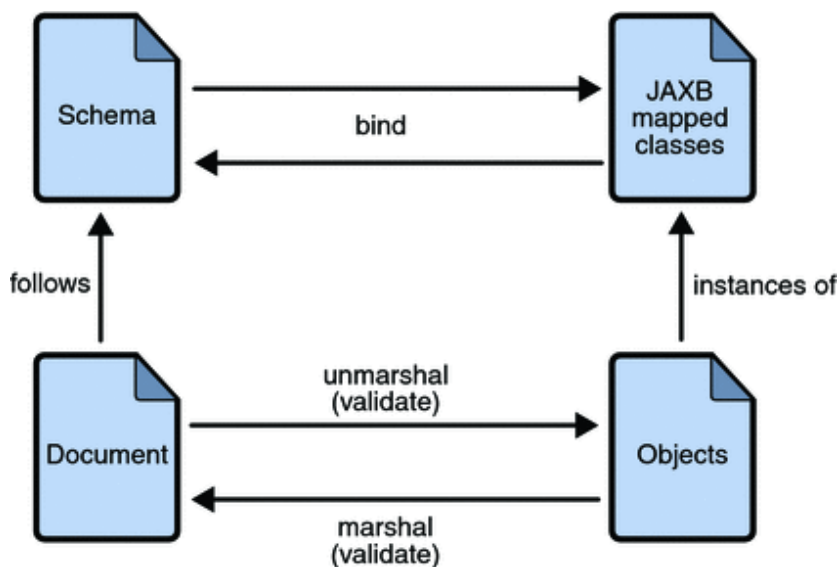


Figure 1 Steps in the JAXB Binding Process [4].

The JAXB binding compiler (xjb) allows for the generation of Java classes from a given XML schema (xsd). The JAXB binding compiler transforms an XML schema into a collection of Java classes that match the structure described in the XML schema. The JAXB generated classes are annotated with special JAXB annotations, in turn the afore mentioned annotations provide a framework to process the XML document

The JAXB compiler binds an XML schema to a set of Java classes. An XML schema is an XSD document that describes, very precisely, the elements and attributes authorized in a certain type of XML

document. The example below shows the shifts section from the Nurse Rostering Competition data set

```
<ShiftTypes>
  <Shift ID="E">
    <StartTime>06:30:00</StartTime>
    <EndTime>14:30:00</EndTime>
    <Description>Early</Description>
    <Hours>8</Hours>
    <Skills>
      <Skill>Nurse</Skill>
    </Skills>
  </Shift>
  <Shift ID="L">
    <StartTime>14:30:00</StartTime>
    <EndTime>22:30:00</EndTime>
    <Description>Late</Description>
    <Hours>8</Hours>
    <Skills>
      <Skill>Nurse</Skill>
    </Skills>
  </Shift>
  <Shift ID="D">
    <StartTime>08:30:00</StartTime>
    <EndTime>16:30:00</EndTime>
    <Description>Day shift</Description>
    <Hours>8</Hours>
    <Skills>
      <Skill>Nurse</Skill>
    </Skills>
  </Shift>
  <Shift ID="N">
    <StartTime>22:30:00</StartTime>
    <EndTime>06:30:00</EndTime>
    <Description>Night</Description>
    <Hours>8</Hours>
    <Skills>
      <Skill>Nurse</Skill>
    </Skills>
  </Shift>
</ShiftTypes>
```

Code 06: XML shift types example.

The corresponding XML schema describes the ShiftTypes Shifts, which contain details of the shift start and end times, skills required, and hours worked. An XML schema description is extremely rigorous and can include details such as the number of elements allowed in a list of objects (cardinality), optional and mandatory attributes, custom data types and more. Below is the schema for this section of the Nurse Rostering Competition data set:

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:complexType name="ShiftTypes">
  <xs:sequence>
    <xs:element name="Shift" maxOccurs="unbounded">
      <xs:complexType>
        <xs:all>
          <xs:element name="StartTime" type="xs:time" />
          <xs:element name="EndTime" type="xs:time" />
          <xs:element name="Description" type="xs:string" minOccurs="0" />
          <xs:element name="Hours" type="xs:float" minOccurs="0" />
          <xs:element name="Skills" minOccurs="0">
            <xs:complexType>
              <xs:sequence>
                <xs:element name="Skill" type="ID" maxOccurs="unbounded" />
              </xs:sequence>
            </xs:complexType>
          </xs:element>
        </xs:all>
        <xs:attribute name="ID" type="ID" use="required" />
      </xs:complexType>
    </xs:element>
  </xs:sequence>
```

</xs:complexType>

Code 07: XSD shift types example.

To run the JAXB compiler from the command line against the schema, run the following command:

```
$xjc [name.xsd] -p [Java Package] -d [Output Directory]
```

XML and other mark-up languages are all forms of semi-structured data. OEM (Object Exchange Model) [1] was created prior to XML as a means of self-describing a data structure. XML has been popularized by web services that are developed utilizing SOAP principles [2].

There are a number of reasons for using XML in an application. Some of the main reasons are listed as follows:

XML is a widely accepted open standard.

XML the clear separation of content from form (appearance).

XML is text-oriented.

XML is extensible.

XML is self-describing.

XML is universal; meaning internationalization is no problem.

XML is independent from platforms and programming languages.

XML provides a robust and durable format for information storage.

XML is easily transformable.

XML is a future-oriented technology.

XML schemas (xsd) describe a certain set of standards for a document, so for businesses to communicate structured data in a standard way, the process is simplified greatly by just following the standards laid down in the schema. One of the main benefits to this approach is two pieces of software can communicate without knowing anything about each other and have to only adhere to a common schema.

AJAX

XML is widely used on the internet to deliver dynamic content to web content via the XMLHttpRequest protocol. This is generally referred to as AJAX (Asynchronous JavaScript and XML). The process of AJAX is that some event on the page triggers an XMLHttpRequest (AJAX) from an external server, the server responds with an XML feed that is passed at the client. JavaScript is used to take values from the feed and the result is inserted in to the web page without the need for a page reload.

XML is also used in software application configuration files to store settings, maintain history, store user setting and many more. The flexibility of XML as well as being self-documenting and human readable, makes XML ideally suited to this task.

XHTML

- XHTML stands for Extensible Hypertext Markup Language. It is the next step in the evolution of the internet. The XHTML 1.0 is the first document type in the XHTML family.
- XHTML is almost identical to HTML 4.01 with only few differences. XHTML is a cleaner and stricter version of HTML 4.01.

- XHTML documents are XML conforming as they are readily viewed, edited, and validated with standard XML tools.
- XHTML documents can be written to operate better than they did before in existing browsers as well as in new browsers.
- XHTML documents can utilize applications such as scripts and applets that rely upon either the HTML Document Object Model or the XML Document Object Model.
- XHTML has stricter syntax rules in comparison to HTML. XHTML gives you a more consistent, well-structured format so that your webpages can be easily parsed and processed by present and future web browsers. You can easily maintain, edit, convert and format your document in the long run.
- Since XHTML is an official standard of the World Wide Web Consortium (W3C)[3], your website becomes more compatible with many browsers and it is rendered more accurately. XHTML combines strength of HTML and XML. Also, XHTML pages can be rendered by all XML enabled browsers.

3.3 Employee rostering schema (XSD)

The employee rostering schema contains all the settings needed to generate a roster for the Employee Web Application; this includes lists of employees, shifts, skills, and also contract information like the maximum number of days to work in a given period and, lastly, day and shift off requests.

Describe each tag element in the XDS file.

Listed below is the first element in the employee xsd. This element describes the root node which, in this case, is SchedulingPeriod

```
<xs:element name="SchedulingPeriod">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="StartDate" type="xs:date" />
      <xs:element name="EndDate" type="xs:date" />
      <xs:element name="Schedule" type="Schedule" minOccurs="0" />
      <xs:element name="Skills" type="Skills" minOccurs="0" />
      <xs:element name="ShiftTypes" type="ShiftTypes" />
      <xs:element name="Patterns" type="Patterns" minOccurs="0" />
      <xs:element name="Contracts" type="Contracts" />
      <xs:element name="Employees" type="Employees" />
      <!-- Added for Location -->
      <xs:element name="Patients" type="Patients" />
      <!-- End -->
      <xs:element name="CoverRequirements" type="CoverRequirements" />
      <xs:element name="DayOffRequests" type="DayOffRequests" minOccurs="0" />
      <xs:element name="DayOnRequests" type="DayOnRequests" minOccurs="0" />
      <xs:element name="ShiftOffRequests" type="ShiftOffRequests" minOccurs="0" />
      <xs:element name="ShiftOnRequests" type="ShiftOnRequests" minOccurs="0" />
    </xs:sequence>
    <xs:attribute name="ID" type="xs:string" use="required" />
    <xs:attribute name="OrganisationID" type="xs:string" use="optional" />
  </xs:complexType>
</xs:element>
```

Code 08: XSD SchedulingPeriod.

This part of the xsd enforces that the root element SchedulingPeriod will have two attributes and fourteen child elements. The most basic XML that would conform to this part of the schema would be

```
<SchedulingPeriod ID="" xmlns:xsi="">
  <StartDate/>
  <EndDate/>
  <Schedule/>
  <Skills/>
  <ShiftTypes/>
  <Patterns/>
  <Contracts/>
  <Employees/>
  <Patients/>
  <CoverRequirements/>
  <DayOffRequests/>
  <DayOnRequests/>
  <ShiftOffRequests/>
  <ShiftOnRequests/>
</SchedulingPeriod>
```

Code 09: XSD Element order.

The next two sections of the above XML are direct children of the root element and have no attributes so the first section of the xsd file covers these. The next element schedule is a child of SchedulingPeriod and is classed as a complexType. Below is the xsd for schedule

```
<xs:complexType name="Schedule">
  <xs:sequence>
    <xs:element name="Assignment" minOccurs="0" maxOccurs="unbounded">
      <xs:complexType>
        <xs:sequence>
          <xs:element name="Date" type="xs:date"/>
          <xs:element name="Employee" type="xs:string"/>
          <xs:element name="ShiftType" type="xs:string"/>
        </xs:sequence>
      </xs:complexType>
    </xs:element>
  </xs:sequence>
</xs:complexType>
```

Code 10: XSD Schedule.

The schema snippet above states that the schedule element must have at least one child named Assignment and this element must have three child elements namely Date, Employee and ShiftType. The XML below would conform to this schema snippet

```
<Schedule>
  <Assignment>
    <Date/>
    <Employee/>
    <ShiftType/>
  </Assignment>
</Schedule>
```

Code 11: XML Schedule minimum requirements.

The full schema (xsd) and XML are in the appendix for further examination

3.4 Testing JAXB Classes (output data to console)

The following screen shot demonstrates output values from the Java classes created from the definition XML document. The location values are stored as follows in the input XML

```
<Employee ID="0">
  <ContractID>0</ContractID>
```



```

    <Name>0</Name>
    <Cost>6</Cost>
    <Hours>8</Hours>
    <Locations>
        <Location lat="53.348654" lon="-2.121572" name="Poynton"/>
    </Locations>
    <Skills>
        <Skill>Nurse</Skill>
    </Skills>
</Employee>

```

Code 12: XML Employee.

The Java code below connects to the definition XML document and retrieves the values as follows

```

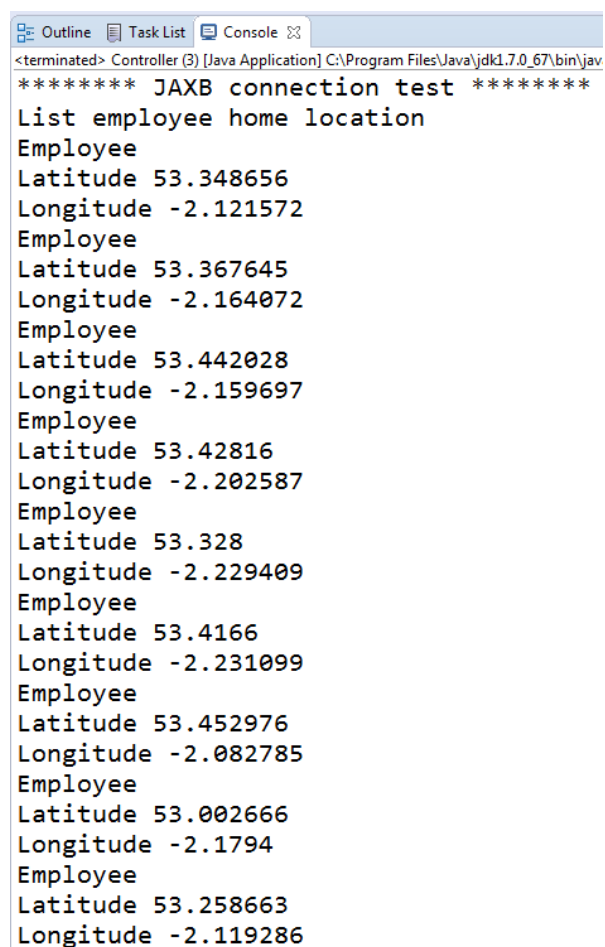
JAXBContext jaxbContent = JAXBContext.newInstance(ObjectFactory.class);
Unmarshaller u = jaxbContent.createUnmarshaller();
sp = (SchedulingPeriod) u.unmarshal(new File(inputXML));

System.out.println("***** JAXB connection test *****");
System.out.println("List employee home location");
for(Employee e : sp.getEmployees().getEmployee()){
    System.out.println("Employee");
    System.out.println("Latitude "+e.getLocations().getLocation().getLat());
    System.out.println("Longitude "+e.getLocations().getLocation().getLon());
}

```

Code 13: Java code JAXB connection test.

The console output



```

<terminated> Controller (3) [Java Application] C:\Program Files\Java\jdk1.7.0_67\bin\jav
***** JAXB connection test *****
List employee home location
Employee
Latitude 53.348656
Longitude -2.121572
Employee
Latitude 53.367645
Longitude -2.164072
Employee
Latitude 53.442028
Longitude -2.159697
Employee
Latitude 53.42816
Longitude -2.202587
Employee
Latitude 53.328
Longitude -2.229409
Employee
Latitude 53.4166
Longitude -2.231099
Employee
Latitude 53.452976
Longitude -2.082785
Employee
Latitude 53.002666
Longitude -2.1794
Employee
Latitude 53.258663
Longitude -2.119286

```

Figure 2 Console JAXB test output.

3.5 Extend schema to include location data and patients

As the initial definition XML document and schema do not support patients and visit location the initial schema has been extended to include employee start location and a list of patients have been added, this involved adding a new patients element to the schema that contains patient elements and adding a locations element to each employee element. There has also been a custom datatype added WeekDay this will be reflected in the created Java classes.

Additions of the location element to each employee element to the schema

```
<!-- Added for employee start location -->
<xs:element name="Locations">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="Location">
        <xs:complexType>
          <xs:simpleContent>
            <xs:extension base="xs:string">
              <xs:attribute type="xs:float" name="lat" use="required"/>
              <xs:attribute type="xs:float" name="lon" use="required"/>
              <xs:attribute type="xs:string" name="name" use="required"/>
            </xs:extension>
          </xs:simpleContent>
        </xs:complexType>
      </xs:element>
    </xs:sequence>
  </xs:complexType>
</xs:element>
<!-- End -->
```

Code 14: Added for employee start location.

Addition of the Patients element to the schema

```
<!-- Added for location of Patients -->
<xs:complexType name="Patients">
  <xs:sequence>
    <xs:element name="Patient" maxOccurs="unbounded">
      <xs:complexType>
        <xs:sequence>
          <xs:element name="Days">
            <xs:complexType>
              <xs:sequence>
                <xs:element name="Day" maxOccurs="unbounded">
                  <xs:complexType>
                    <xs:simpleContent>
                      <xs:extension base="xs:string">
                        <xs:attribute type="Time" name="Time"
use="required"/>
                        <xs:attribute type="WeekDay" name="Day"
use="required"/>
                      </xs:extension>
                    </xs:simpleContent>
                  </xs:complexType>
                </xs:element>
              </xs:sequence>
            </xs:complexType>
          </xs:element>
          <xs:element type="xs:string" name="Name"/>
          <xs:element type="xs:string" name="HouseNumber"/>
          <xs:element type="xs:string" name="Street"/>
          <xs:element type="xs:string" name="Area"/>
          <xs:element type="xs:string" name="Postcode"/>
          <xs:element name="Location">
            <xs:complexType>
              <xs:simpleContent>
                <xs:extension base="xs:string">
                  <xs:attribute type="xs:float" name="lat" use="required"/>
                  <xs:attribute type="xs:float" name="lon" use="required"/>
                </xs:extension>
              </xs:simpleContent>
            </xs:complexType>
          </xs:element>
        </xs:sequence>
      </xs:complexType>
    </xs:element>
  </xs:sequence>
</xs:complexType>
```

```

        </xs:simpleContent>
      </xs:complexType>
    </xs:element>
  </xs:sequence>
  <xs:attribute type="ID" name="ID" use="required"/>
</xs:complexType>
</xs:element>
</xs:sequence>
</xs:complexType>
<!-- End -->

```

Code 15: Addition of patient element.

Addition custom data type

```

<xs:simpleType name="WeekDay">
  <xs:restriction base="xs:string">
    <xs:enumeration value="Sunday" />
    <xs:enumeration value="Monday" />
    <xs:enumeration value="Tuesday" />
    <xs:enumeration value="Wednesday" />
    <xs:enumeration value="Thursday" />
    <xs:enumeration value="Friday" />
    <xs:enumeration value="Saturday" />
    <!-- Added for location -->
    <xs:enumeration value="All" />
    <!-- End -->
  </xs:restriction>
</xs:simpleType>

```

Code 16: XSD Custom datatype.

3.6 Summary

In this chapter we have covered the input XML, XSD schema and the use of JAXB to bind values from a XML document to Java classes that can then be used in a number of java packages including servlets. JAXB is native to the standard Java package and can be used without any add-ons. JAXB is an industry standard method for binding to XML sources.

Chapter 4 Design and Implementation of the Workforce Scheduling Software

4.1 Introduction

This chapter describes the algorithm selection based on results of Berlin 52 test results, the creation of a schedule based on an XML definition document, the assignment of days to the schedule and the assignment of employees and shifts to each day.

The minimum criteria to create an initial feasible solution which conforms to the listed hard constraints. The application of metaheuristic algorithms to compare generated schedules against a list of soft constraints to find the optimal schedule that satisfy all hard constraints and as few as possible of all soft constraints.

Extending the Workforce Scheduling Software to be location aware and cluster in-home patient visits to nurse start of work location.

4.2 Design of the Workforce Scheduling Software

The design of the Workforce scheduling web application has been split in to, two java servlets, one connects to definition XML document, retrieves the setting to create the schedule we are going to optimise, these setting are passed to a java optimiser that has facility to use a selection of different optimisation algorithm's to find the best feasible solution in a reasonable amount of time.

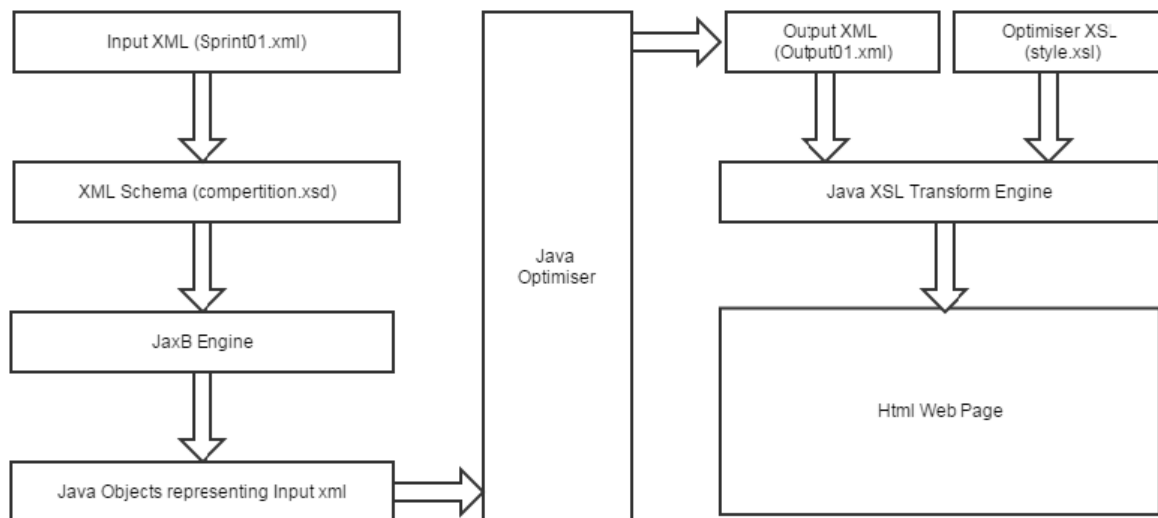


Figure 3 Overall program flow.

Each time the optimiser finds a better solution an output.xml document is created which is passed to a front end processor which applies a XSL transform to the output.xml which then renders to a web page via AJAX

4.3 Implementation of the Workforce Scheduling Software

The implementation of the Workforce Scheduling Web Application has been developed in Java. The workforce scheduling application will be hosted on a virtual private server. A Tomcat Version 7 server will be installed to serve the web application to the internet via Nginx web server which will

offer some security to the web application as well as offering the ability to use URL rewriting which offers friendly URLs.

Two levels of security will be employed

All traffic to the website will be served via https.

Two levels of password will be used the first a user level which will give user access to their own schedule and an admin password which will be needed to access the optimiser and administration pages and will also offer access to all users schedules.

The web application will consist of three pages a user page where each employee can see their individual schedule for the given schedule and also any soft constraints violations that are relevant to them.

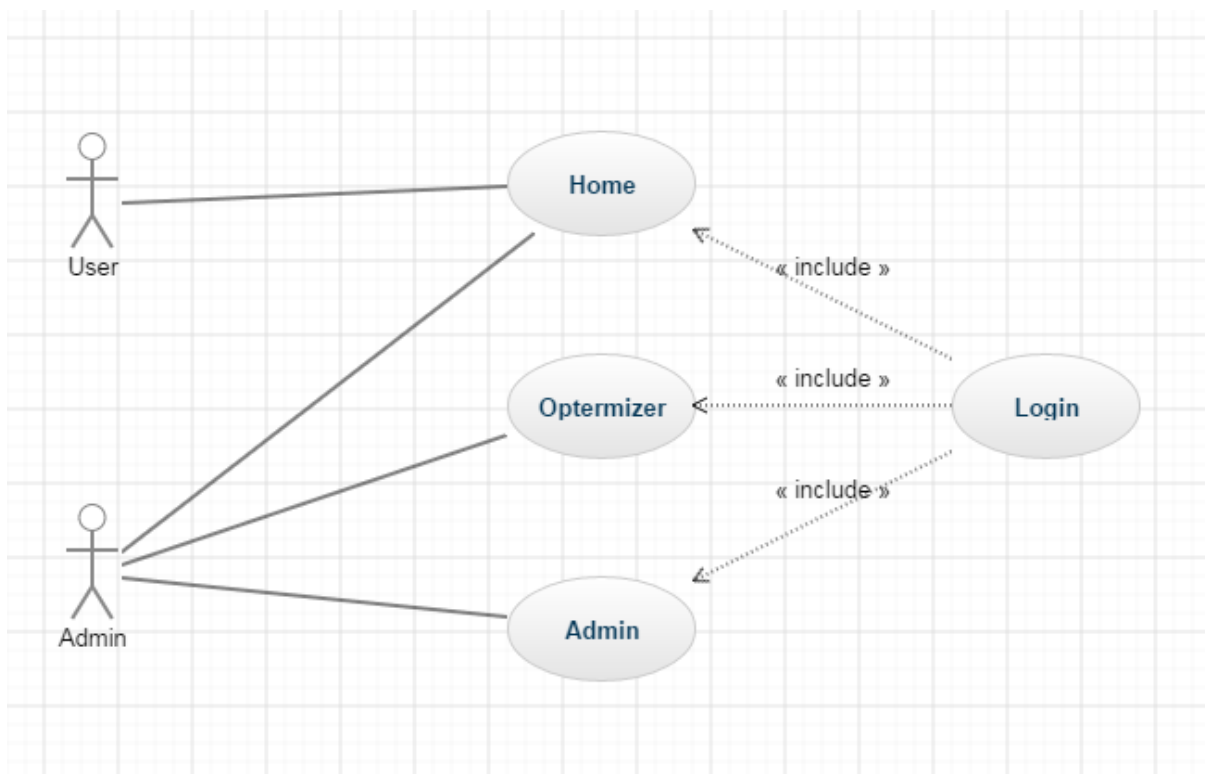


Figure 4 Use Case Diagram.

4.3.1 Initialise schedule with feasible solution

When the optimizer is initiated a Solution object is created which contains the schedule. JAXB is used to create classes containing values bound from an definition XML document, a number of objects are used to create an initial feasible schedule which are described below.

SchedulingPeriod object which supplies the start and end dates for the schedule and also the cover requirements shifts of each type are required per day.

Solution object which holds the created schedule, Solution is created from SolutionFactory, in the this case the constructed object concrete class is NRP this does allow for different types of the Solution in the future and is a use of the Factory Design Pattern.

Day object holds a list of available Employees objects and a list of Shift objects that must be covered according to the cover requirements stored in the SchedulingPeriod object.

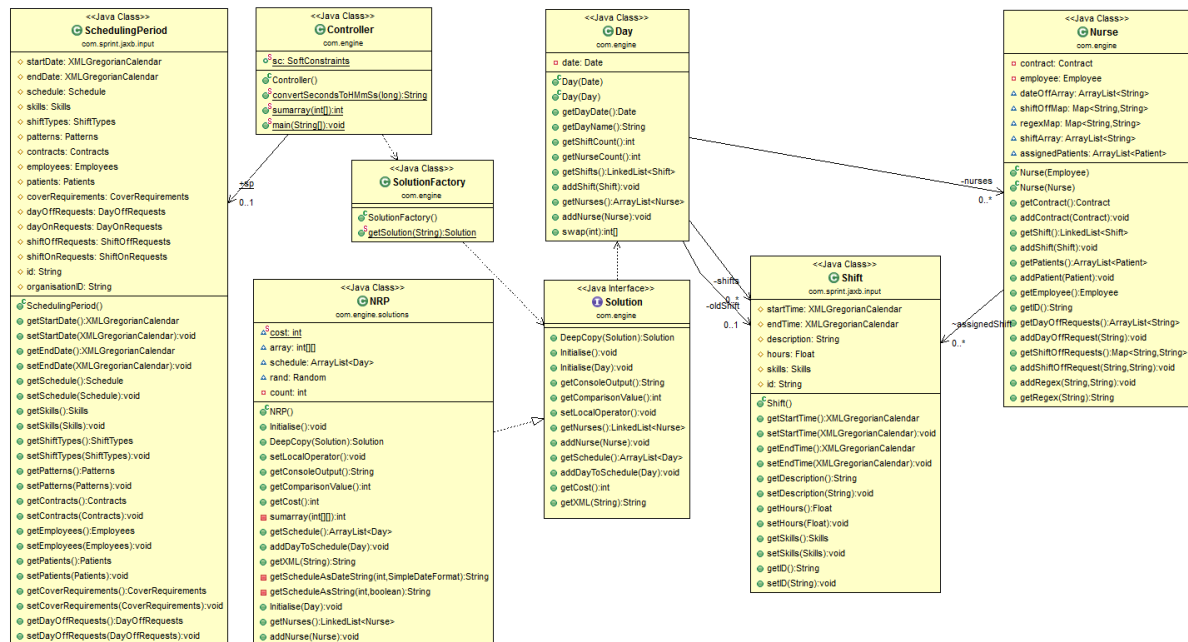


Figure 5 UML Initialise Feasible Schedule.

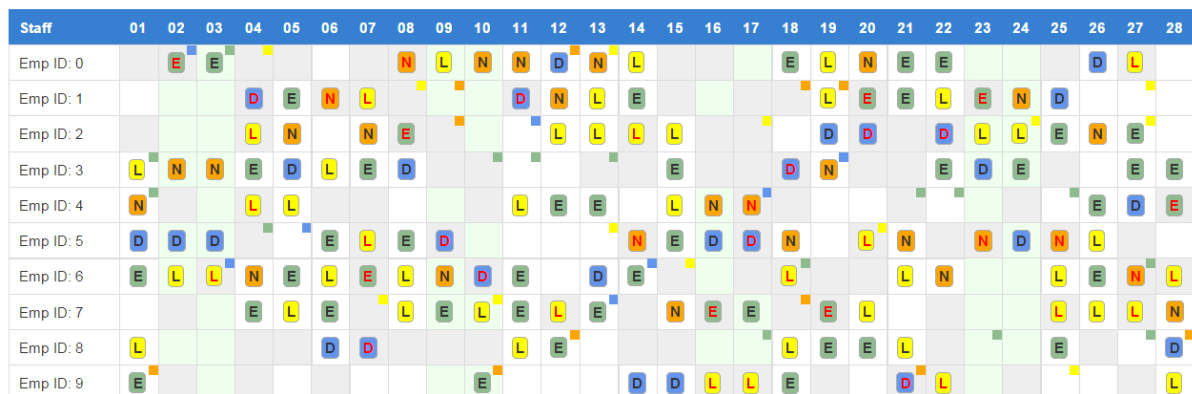
Once you run `Solution.Initialise()` the Shift objects stored in each day are assigned to the Nurses objects stored in each day object, this is achieved by conforming to two hard constraints which are;

All shifts for each day must be assigned.

Only one shift can be assigned to one nurse per day.

The result of this is a feasible schedule where each day has list of employees with shifts assigned to them, that in first place respects the cover requirements needed, and in the second place conforms to the hard constraints listed earlier.

January



Input XML: sprint01.xml Soft Constraint Violations: 72 Algorithm: ILS

■ Early Shift
■ Day Shift
■ Late Shift
■ Night Shift

Figure 6 Initialised Feasible Schedule.

4.3.2 Soft Constraint Violations

Soft constraint violations do not make the schedule unfeasible but result in a less optimised schedule. Soft constraints violations are summed and a integer is returned the lower the integer the more optimal the schedule is. The following soft constraints will be used to compare for violations.

4.3.2.1 Soft Constraints that are standard across all Employees

DayOffRequests

Days that each employees may of asked not to work in the planning horizon.

ShiftOffRequests

Shifts that each employees may of asked not to work in the planning horizon

4.3.2.2 Soft Constraints that are specific to an Employees contract

Patterns

Shift patterns not to be worked for instance no early shift following a night shift

MaxNumAssignments

The maximum amount of days that the employee may work in the planning horizon.

MinNumAssignments

The minimum amount of days that the employee may work in the planning horizon.

MaxConsecutiveWorkingDays

The maximum amount of consecutive working days to be worked in the planning horizon.

MinConsecutiveWorkingDays

The minimum amount of consecutive working days to be worked in the planning horizon.

MaxConsecutiveFreeDays

The maximum amount of consecutive days off in the planning horizon.

MinConsecutiveFreeDays

The minimum amount of consecutive days off in the planning horizon.

CompleteWeekends

A weekend is defined in the problem definition and can be sat - sun, fri - sat - sun, sat - sun - mon or fri - sat - sun - mon, this violation states that if you are working part of the weekend as defined in the definition problem the employee works all weekend shifts.

IdenticalShiftTypesDuringWeekend

This soft constraint states that all shift types for the weekend as defined by the definition problem are of the same type.

The Solution object uses the SoftConstraints object that tests for soft constraint violations, Solution.getComparisonValue() tests each soft constraint against the SoftConstraints object and keeps a running total of any violations, after all tests are completed the total of all violations is returned as an integer.

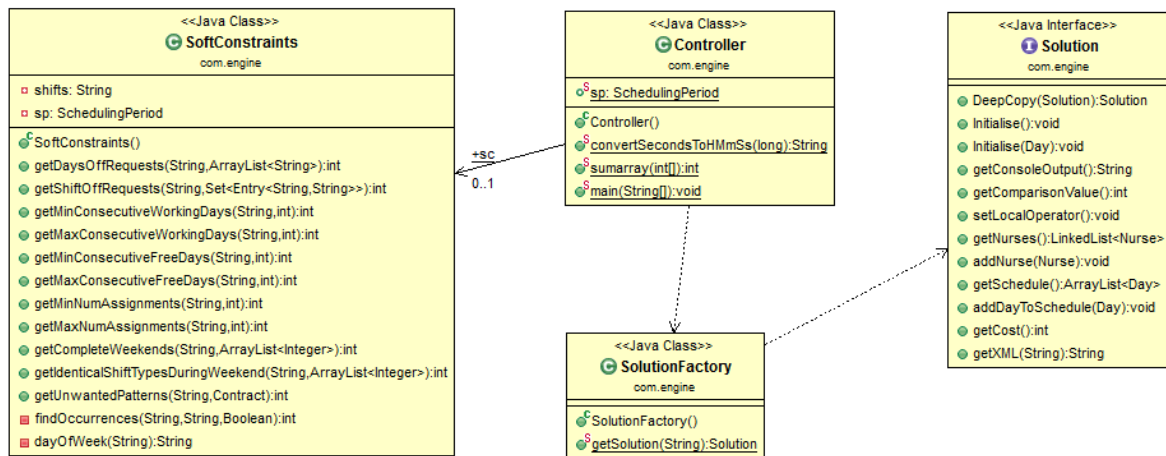


Figure 7 UML Soft Constraint Violations

4.3.2.3 Soft Constraint violation detection

To detect whether a soft constraint has been violated each employee in the schedule a snap shot of their planning horizon is converted to a string, then any blank shifts are substituted by a "X". The screen shot Fig 08 shows the selection that will be converted to a string.

January

Staff	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
Emp ID: 0		E	E					N	L	N	N	D	N	L				E	L	N	E	E				D	L	
Emp ID: 1				D	E	N	L					D	N	L	E				L	E	E	L	E	N	D			
Emp ID: 2				L	N		N	E				L	L	L	L				D	D		D	L	L	E	N	E	
Emp ID: 3	L	N	N	E	D	L	E	D							E			D	N			E	D	E			E	E
Emp ID: 4	N			L	L						L	E	E		L	N	N									E	D	E
Emp ID: 5	D	D	D			E	L	E	D					N	E	D	D	N		L	N		N	D	N	L		
Emp ID: 6	E	L	L	N	E	L	E	L	N	D	E		D	E				L			L	N			L	E	N	L
Emp ID: 7				E	L	E		L	E	L	E	L	E		N	E	E		E	L					L	L	L	N
Emp ID: 8	L					D	D					L	E					L	E	E	L				E			D
Emp ID: 9	E									E				D	D	L	L	E			D	L						L

Input XML: sprint01.xml Soft Constraint Violations: 72 Algorithm: ILS

■ Early Shift
■ Day Shift
■ Late Shift
■ Night Shift

Figure 8 Conversion of employee schedule to string.

This conversion process would result in a string as follows "XXXLNXNEXXXLLLLXXXDDXDLEENEX". This string is then passed by a soft constraint violation engine where regular expression is used to determine if it has violated the soft constraint.

Example of regular expression for maximum number of assignments in planning horizon

```

public int getMaxNumAssignments(String s, int i){
    i++;
    String regex = "([E|D|L|N])";
    return (i>0)?(findOccurrences(regex,s,false)>=i)?1:0:0;
}
  
```

Code 18: Java regular expression.

The example above reads the value i , if the occurrences of E,D,L or N in the input string regex are equal or greater than i the soft constraint is reported as violated.

There are similar regular expression checks for each soft constraint.

4.3.3 Local Operators

The local operator used for this project is a single random swap, this means that of all the days stored in schedule a random one is selected and of the available employees for the selected day two are randomly selected and the shifts swapped.

Staff	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
Emp ID: 0						L	N				N	L	E	E	E	D	D	N		N	E	D	E	E				
Emp ID: 1	N	D	D	N				D	D	D			E	N					D	N	N				E	E	E	E
Emp ID: 2					L	N	D					D	L	L	E			D	D	E		E	N	N	E		D	D
Emp ID: 3	D	N	N	E	L			L	E			D	E					E	E			L	L	L			N	N
Emp ID: 4				E	D			E	N				L	E	D				L	L					L	E	L	
Emp ID: 5	E	L	L						L	L	L	E	E		L	N	N	L	E		D	E				L	L	L
Emp ID: 6				L	E	E							D	L						L	L				D	L		
Emp ID: 7	E			L	E	D			E	N	N	L			L	L	L			E	L		D	D		D	E	L
Emp ID: 8	L					E	L				L	L						E	N						L			
Emp ID: 9	L	E	E	D	N	L	E	L	E	E	E	N	N	D	N	E	E	L	L	L	E				N	N		E
Optimize Day	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA

Figure 9 Swapping two shifts with single random swap Local Operator.

So for the definition XML being used, sprint_location_01.xml this means there will be 28 days in the schedule with ten available employees per day. The cover requirements state each weekday be assigned six shifts of four types and each weekend day assigned four shifts of four types.

The single random swap local operator will randomly select one day from the schedule, from this day which has ten available employees stored even thou only six of them will have shifts assigned, if this is a weekday or four if a weekend day, two employees will be randomly selected and the shifts will be swapped.

This results of applying this local operator is a slightly altered schedule which may be more or less optermized than the previous schedule.

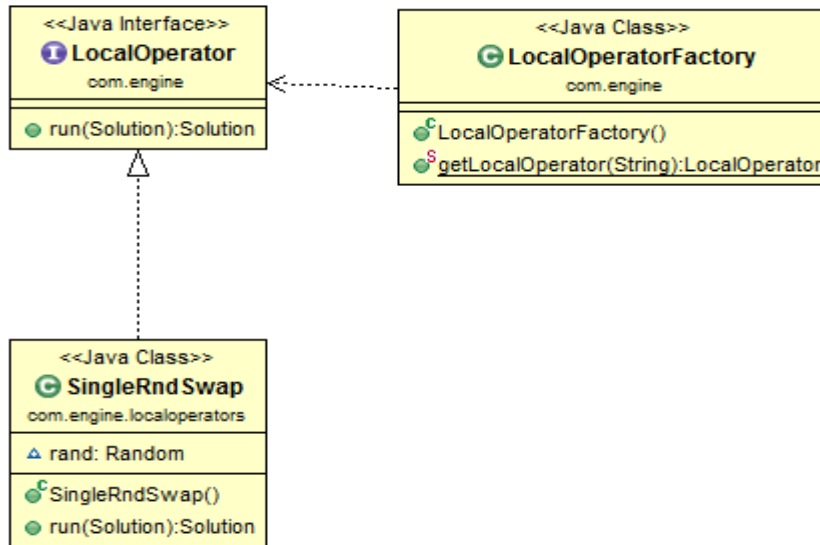


Figure 10 UML Local Operator.

4.3.4 Applying Metaheuristics

Two metaheuristics algorithms are available to optimize the nurse rostering schedule, these algorithms have been chosen from the most popular, and gave the best results in the Berlin52 tests after one thousand passes.

This code creates and initialize a feasible solution which then uses one of the metaheuristics described below to find the optermized schedule.

```

// Initialise Nurse Rostering Problem Solution
Solution solution = SolutionFactory.getSolution("NRP");
// Initialise metaheuristics to be used
Algorithm algorithm = AlgorithmFactory.getAlgorithm(algo);

// Initialise Nurse Rostering Problem Solution
try {
    solution.Initialise();
} catch (ParseException e) {
    e.printStackTrace();
}
// Optermized Solution
Solution test = solution.DeepCopy(solution);
optermized = solution.DeepCopy(algorithm.optimise(test, "SingleRndSwap", "output01.xml"));

```

Code 19: Java feasible solution code.

4.3.4.1 Simulated Annealing

The simulated annealing algorithm was originally taken from the process of annealing in metal work. Annealing involves heating and cooling a metals to alter its physical properties due to the changes in its internal structure. When the metal cools the new structure becomes fixed, causing the metal to retain its new properties.

Simulated annealing uses a temperature variable to simulate the heating process. The variable is slowly reduced to “cool” the system. When the variable is at a high value the algorithm will sometimes except worse solutions, this gives the algorithm the ability to escape local optimums. As the temperature is reduced the chance of accepting a worse solutions is reduced therefore allowing the algorithm to gradually focus in on an optimum solution.

Simulated annealing has a number values which need to be set before optimization, adjustment of these parameters may result in a more optimal solution being found. Adjustment of these parameters is known as tuning the algorithm, the tuning parameters are listed below.

Initial temperature, Cooling Rate and Cooling Curve, tuning (adjustment) of these parameter for a given optimization problem can greatly improve the results.

Simulated Annealing

Definition XML	Max Iterations	Initial Temp	Cooling Rate	Cooling Curve	
Sprint_Location_01 ▼	30000	1	0.0009	Ramp ▼	Optimize (SA)

Figure 11 Tuning Simulated Annealing.

4.3.4.2 Iterated Local Search

Iterated Local search is similar to hill climbing in that if a better solution is found it is saved as the new best solution, in a wide and varied search space hill climbing can often get stuck in a local optima resulting in a non-optimal solution.

By adding a “kick feature” to hill climbing iterated local search can escape from non-optimal local optima and fully search the planning horizon, giving a much improved final solution. The only value to set when tuning iterated local search is the value at which the “kick” is initiated.

Iterated Local Search

Definition XML	Max Iterations	Max No Improvement	
Sprint_Location_01 ▼	30000	3000	Optimize (ILS)

Figure 12 Tuning Iterated Local search.

4.4 Extending Nurse Rostering Schedule to Cluster patient visits to staff locations

Extending the nurse rostering schedule to include nurses making visits to patients houses. To achieve this some additional information needs to be added to the definition XML and schema XSD, these are described in detail in chapter three.

With the employee home location and a set of patients added to the definition XML a new optimizer and local operator can be added to the project to cluster patient visits around nurse home locations.

The code design of the workforce scheduling system employs the factory design pattern making the addition of these new features as easy as adding two classes, a concrete class to calculate the clustering of patients which can be instantiated from the SolutionFactory using the Solution interface and a concrete class to add a different local operator which can be instantiated from the LocalOperator Factory using the LocalOperator Factory.

```
// Initialise Patient Visit Solution
Solution pv = SolutionFactory.getSolution("PV");
// Initialise metaheuristics to be used
Algorithm algorithm = AlgorithmFactory.getAlgorithm(algo);

// Initialise Patient Visit Solution
try {
    pv.Initialise(optimizer.getSchedule().get(Integer.parseInt(day)));
} catch (NumberFormatException | ParseException e) {
    e.printStackTrace();
}
```

```
// Optimized Solution
Solution pvOptimized = pv.DeepCopy(algorithm.optimise(pv, "SingleRndMove", "location.xml"));
Code 20: Java code location optimizer.
```

4.4.1 Clustering Patients

The clustering of patients to nurse home locations is achieved by first creating a feasible solution which involves randomly assigning all patients for the given day to the available nurses which was calculated in the previous step of the software which calculates the schedule.

Once the feasible solution has been created the overall distance of the solution is measured by reading the value at `Solution.getComparisonValue()`, the solution is slightly altered by applying the new local operator `SingleRndMove`. The two values taken are compared and the lowest value is saved as best known, this is repeated and guided to the optimal solution with use of metaheuristics.

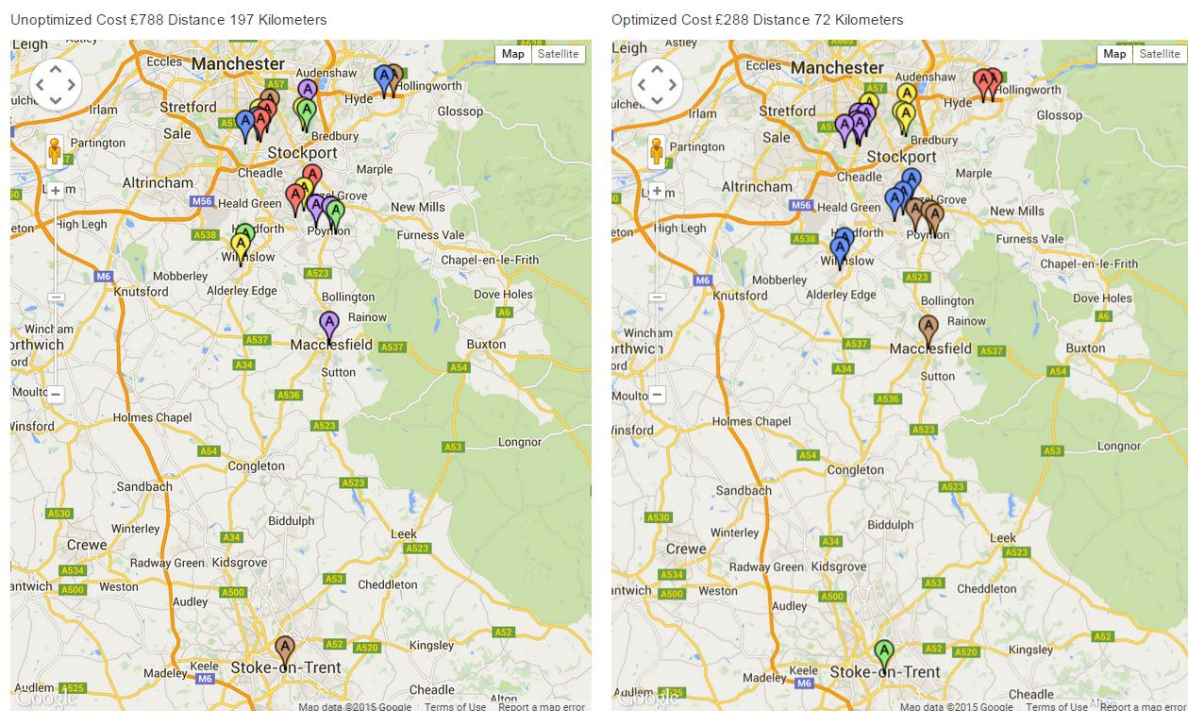


Figure 13 Optimized Clustering of patients.

Each different coloured marker represent a patients assigned to a nurse, by comparing the two pictures above it is clear the optimizer has achieved optimal clustering of patients to nurse start location.

4.4.2 Local Operators

The new local operator is a single random move operator, in effect a random nurse is selected and a patient is removed and then randomly assigned to a new nurse. The reason for a move as appose to a swap is, in the clustering optimization it is not critical that the same amount of patients be assigned to all employees, which is different from the swap operator used in the rostering optimization where two nurses where swapped.

This results of applying this local operator is a slightly altered solution which may have a better or worse overall distance.

4.5 Summary

This chapter has explained the design, extending and implementation of the Workforce Scheduling System. The system has been designed using Java and to run on a Tomcat web server using servlets for client interaction. The servlet design has been split into front end presentation and backend engine that calculates the optimized solutions.

The factory design pattern has been used extensively in the software design allowing for clear and concise design, ease of maintenance and also allowing modification and extension of the software without extensive rewriting.

Feasible schedule creation, initialization, optimization, metaheuristics, local operators have all been explained and screen shots of the client interface showing results have been supplied.

Chapter 5 Evaluation of Workforce Scheduling

5.1 Evaluation of Metaheuristics for WFS

The performance graphs (fitness value vs iteration time) for the SA and ILS algorithms used for nurse rostering are shown in Figs 14 and Fig 15, this a short pass two minute run.

Simulated Annealing

Initial Temperature: 1
Cooling Rate: 0.0009
Cooling Curve: ramp
Stop Condition: Temp = 0 or iterations > 30000

Input XML: sprint_location_01.xml **Soft Constraint Violations:** 39 **Finished Algorithm:** SA

JS charts

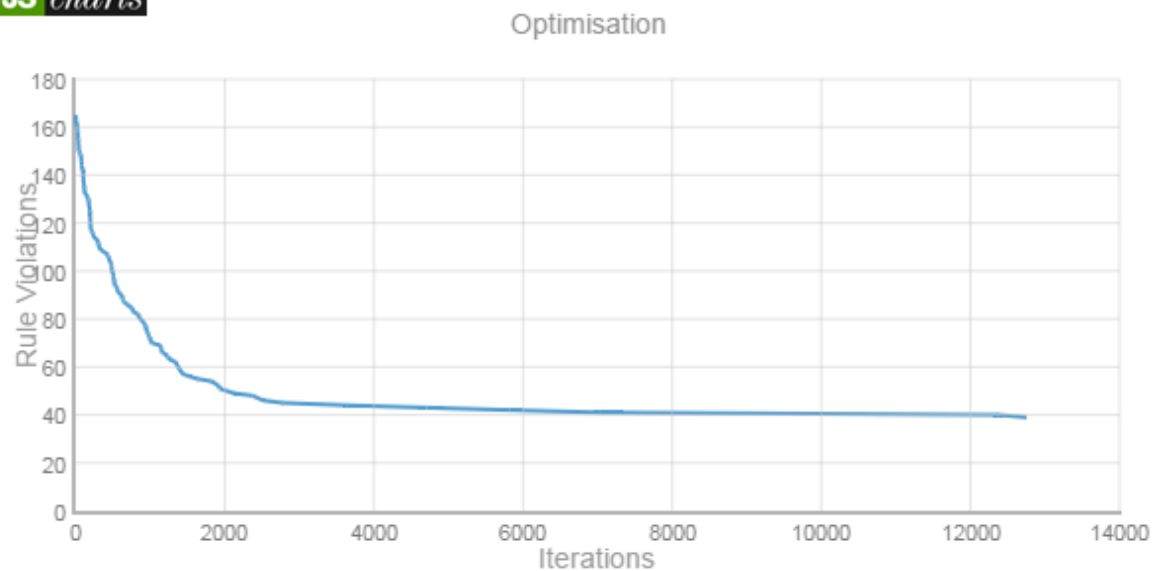


Figure 14 Simulated Annealing two minute pass.

Iterated Local Search

Stop Condition: 30000 iterations

Kick Value: 3000 iterations

Input XML: sprint_location_01.xml **Soft Constraint Violations:** 37 **Finished Algorithm:** ILS

JS charts

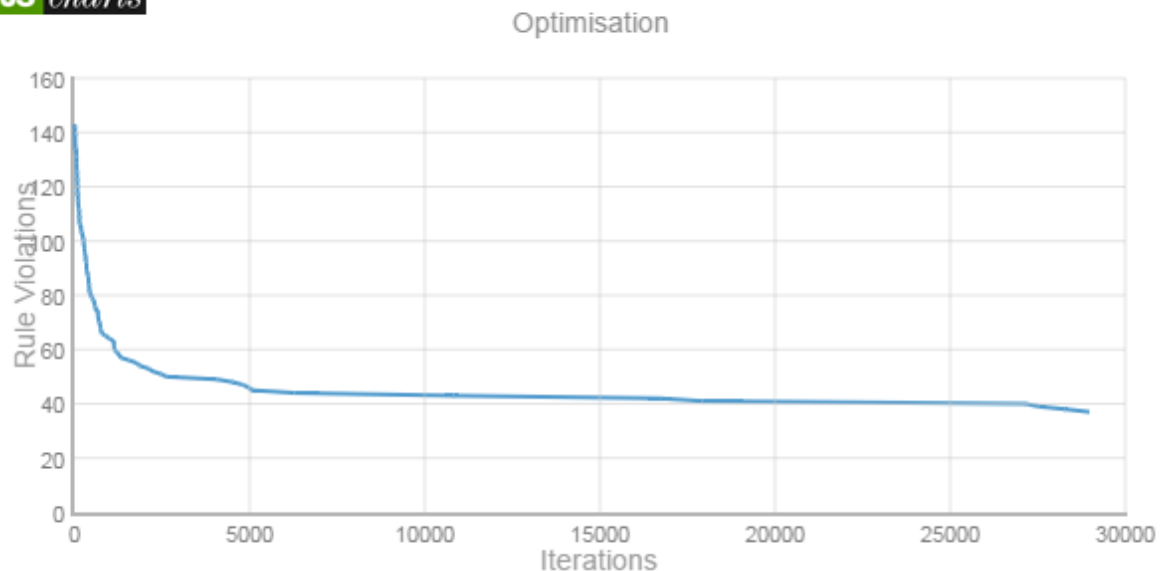


Figure 15 Iterated Local Search two minute pass.

Fig 14 show that Simulated Annealing rapidly descends to a good solution which slowly further improves over time.

Simulated Annealing requires that the following parameters be chosen.

1. Initial temperature
2. Cooling rate
3. Decent equation (linear or ramp)
4. Termination condition

Fig 15 shows that ILS has a slower decent but often reaches the same or better value over time. ILS requires that the following parameters be chosen.

1. Kick value
2. Termination condition

The reduction of parameters to tune makes ILS easier to tune.

Simulated Annealing comparison of different decent curve and cooling rate after twenty passes of each set of parameters.

Best	Average	Initial temperature	Cooling rate	Decent	Termination condition
39	43	1	0.0009	curve	Temp = 0 or iterations > 30000
36	40	1	0.000009	curve	Temp = 0 or iterations > 300000

42	48	1	0.0009	linier	Temp = 0 or iterations > 30000
37	42	1	0.00009	linier	Temp = 0 or iterations > 300000

Table 2 Simulated Annealing comparison of different decent curves

Iterated Local Search comparison of different termination condition and kick value after twenty passes of each set of parameters.

Best	Average	Kick	Termination condition
37	41	iterations > 3000	iterations > 30000
33	38	iterations > 3000	iterations > 300000

Table 3 Iterated Local Search comparison of different termination condition and kick values

5.2 Evaluation of objectives Achieved

5.2.1 Literature review objectives

Objective	Description	Outcome
Scheduling systems	Research existing employee (nurse) scheduling systems	Completed
Nurse Rostering Problem	Research the nurse rostering problem (NRP)	Completed
Competition datasets	Research NRP competition XML datasets and schemas	Completed
XML parsing	Compare Java frameworks for parsing XML	Completed
Trajectory based meta-heuristics	Research trajectory based meta-heuristics for scheduling (e.g. simulated annealing, hill climbing)	Completed
Population based meta-heuristics	Research population based meta-heuristics for scheduling (e.g. genetic algorithms, quantum annealing)	Completed
Web technologies	Research web technologies (Servlets, JSP, Ajax and Server Side Events)	Completed
Virtual servers	Research virtual servers for deploying a nurse scheduling system as a web application	Completed

Table 4 Literature review objectives

5.2.2 Design objectives

Objective	Description	Outcome
Requirements	Perform a requirements analysis	Completed
UML	Undertake a UML design	Completed
UI Design	Undertake user interface (UI) design	Partial

Table 5 Design objectives

5.2.3 Implementation objectives

Objective	Description	Outcome
Java Console	Develop and implement Java console based nurse scheduling system	Completed
Trajectory based optimizers	Develop and implement trajectory based scheduling optimizers (SA, LAHC)	Completed
Manual tuning	Manual tuning the parameters of optimisation algorithms	Completed
VPS deployment	Deploy a nurse scheduling optimizer on a virtual private server	Completed
Administration page	Add an administration page where constraints can be enabled and disabled and day/shift off/on requests, patterns, shifts, skills and employees can be added, edited or deleted	Partial
Configuration page	Add a configuration page where the settings, tuning parameters, and algorithm that will be used to produce the schedule can be adjusted	Completed
Manual adjustment	Add manual adjustment to the completed schedule where a user can fine tune the result by using drag & drop	Partial
Version control	Use version control for project backups	Completed

Table 6 Implementation objectives

5.2.4 Advanced objectives

Objective	Description	Outcome
Alternative optimization	Explore alternative optimization techniques such as iterative local search and multi-neighbourhood search	Completed
Auto tuning	Auto tuning the parameters of optimisation algorithms	Partial
Implement a dispatcher	Implement a dispatcher for assigning tasks to shifts (e.g. using dispatching-rule based methods).	Completed
Optimization of location *	Console application to clustering nurse home location to patient home location	Completed

Table 7 Advanced objectives

* This was the advanced objective chosen so as to implement a home care scheduling solution.

5.3 Usability Testing

5.3.1 Application tested using different browsers on different devices

The Workforce Scheduling Web Application is designed to work on HTML5 compatible browsers, that said the site has been designed using Bootstrap which offers a good degree of compatibility with pre HTML5 browsers

Internet Explorer

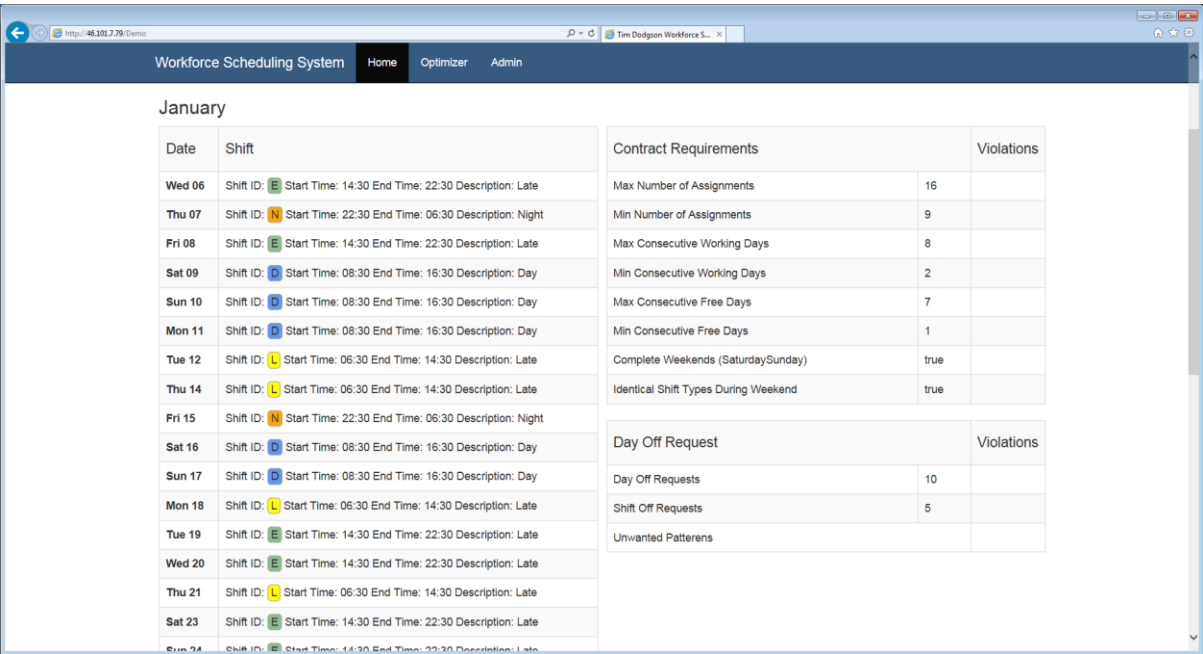


Figure 16 Internet Explorer.

Firefox

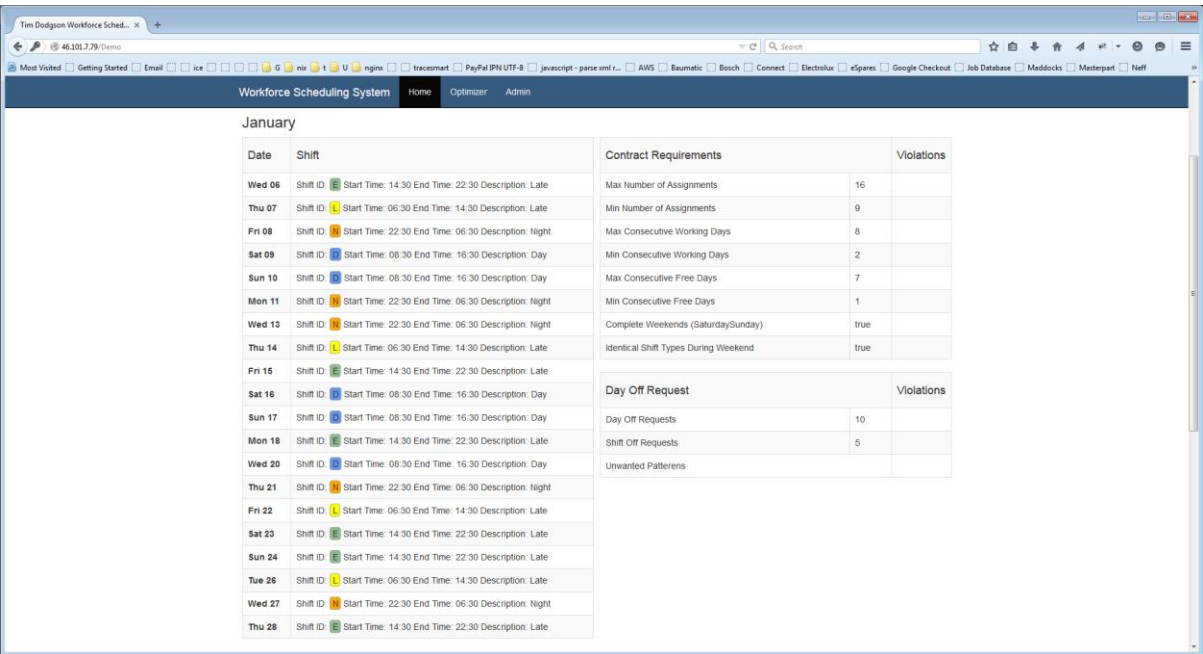


Figure 17 Firefox.

Google Chrome

Date	Shift
Wed 06	Shift ID: E Start Time: 14:30 End Time: 22:30 Description: Late
Thu 07	Shift ID: L Start Time: 06:30 End Time: 14:30 Description: Late
Fri 08	Shift ID: N Start Time: 22:30 End Time: 06:30 Description: Night
Sat 09	Shift ID: D Start Time: 06:30 End Time: 16:30 Description: Day
Sun 10	Shift ID: D Start Time: 06:30 End Time: 16:30 Description: Day
Mon 11	Shift ID: N Start Time: 22:30 End Time: 06:30 Description: Night
Wed 13	Shift ID: N Start Time: 22:30 End Time: 06:30 Description: Night
Thu 14	Shift ID: L Start Time: 06:30 End Time: 14:30 Description: Late
Fri 15	Shift ID: E Start Time: 14:30 End Time: 22:30 Description: Late
Sat 16	Shift ID: D Start Time: 06:30 End Time: 16:30 Description: Day
Sun 17	Shift ID: D Start Time: 06:30 End Time: 16:30 Description: Day
Mon 18	Shift ID: E Start Time: 14:30 End Time: 22:30 Description: Late
Wed 20	Shift ID: D Start Time: 06:30 End Time: 16:30 Description: Day
Thu 21	Shift ID: N Start Time: 22:30 End Time: 06:30 Description: Night
Fri 22	Shift ID: L Start Time: 06:30 End Time: 14:30 Description: Late
Sat 23	Shift ID: E Start Time: 14:30 End Time: 22:30 Description: Late
Sun 24	Shift ID: E Start Time: 14:30 End Time: 22:30 Description: Late
Tue 26	Shift ID: L Start Time: 06:30 End Time: 14:30 Description: Late
Wed 27	Shift ID: N Start Time: 22:30 End Time: 06:30 Description: Night
Thu 28	Shift ID: E Start Time: 14:30 End Time: 22:30 Description: Late

Contract Requirements		Violations
Max Number of Assignments	16	
Min Number of Assignments	9	
Max Consecutive Working Days	8	
Min Consecutive Working Days	2	
Max Consecutive Free Days	7	
Min Consecutive Free Days	1	
Complete Weekends (Saturday/Sunday)	true	
Identical Shift Types During Weekend	true	

Day Off Request		Violations
Day Off Requests	10	
Shift Off Requests	5	
Unwanted Patterns		

Figure 18 Google Chrome.

Mobile Phone

Date	Shift
Fri 01	Shift ID: E Start Time: 14:30 End Time: 22:30 Description: Late
Sat 02	Shift ID: L Start Time: 06:30 End Time: 14:30 Description: Late
Sun 03	Shift ID: L Start Time: 06:30 End Time: 14:30 Description: Late
Mon 04	Shift ID: E Start Time: 14:30 End Time: 22:30 Description: Late
Wed 06	Shift ID: E Start Time: 14:30 End Time: 22:30 Description: Late

Figure 19 iPhone.

5.3.2 Responsive web design (RWD)

Responsive web design is an approach to web design aimed at crafting sites to provide an optimal viewing experience, easy reading and navigation with a minimum of resizing, panning, and scrolling across a wide range of devices (from desktop computer monitors to mobile phones).

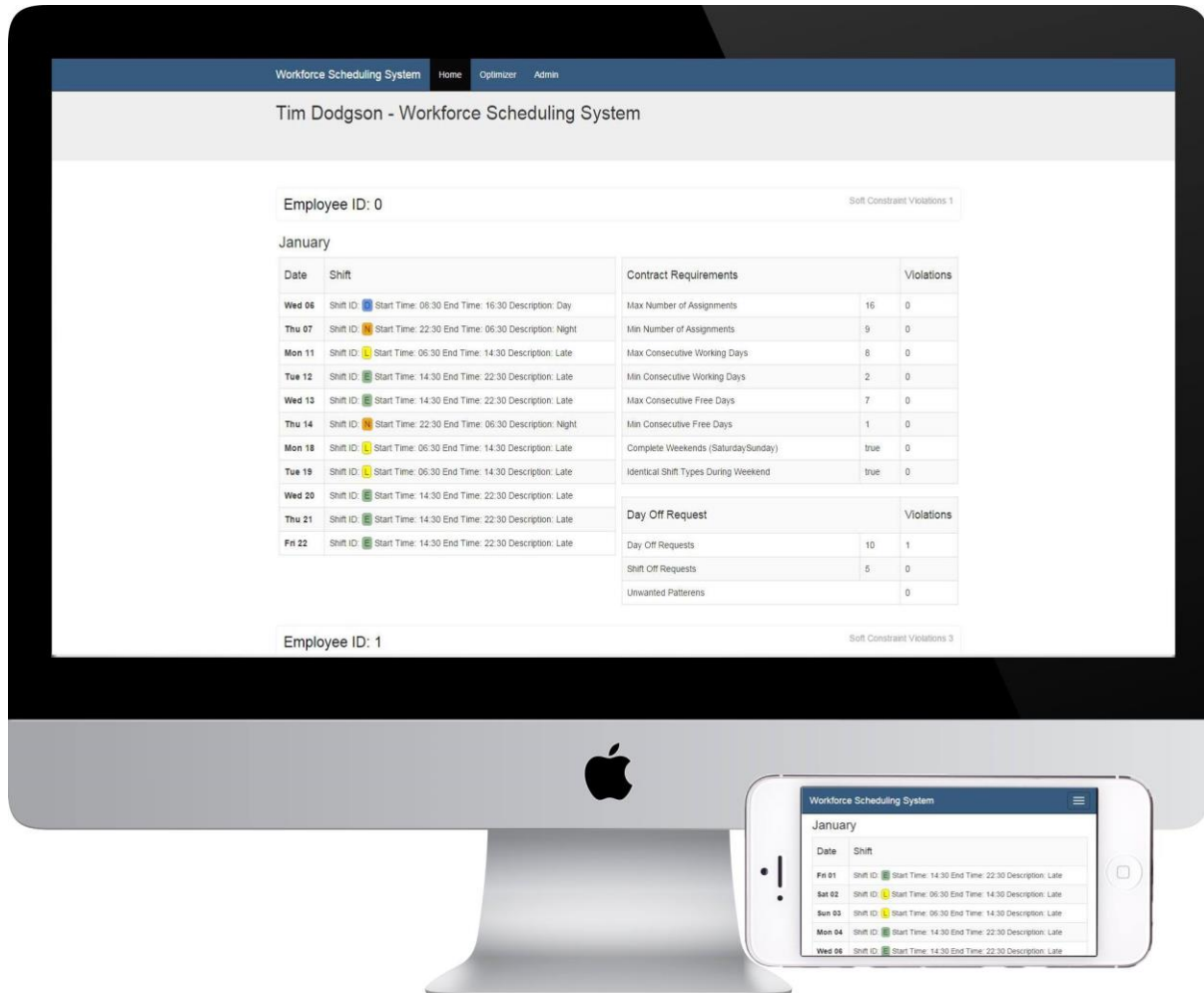


Figure 20 Responsive website design.

5.3.3 Evaluation of Patient to Nurse Clustering

The clustering of patients home location to nurse start of work location evaluation shown in Figure 20, clearly demonstrates that using meta-heuristics to optimize the assignment of patients to nurses

where the distance travelled for each nurse is used to cluster the patients has a dramatic effect on distance travelled.

The clustering of patients in Figure 21 has reduced the overall distance travelled by 125 kilometres which is a sixty three percent improvement.

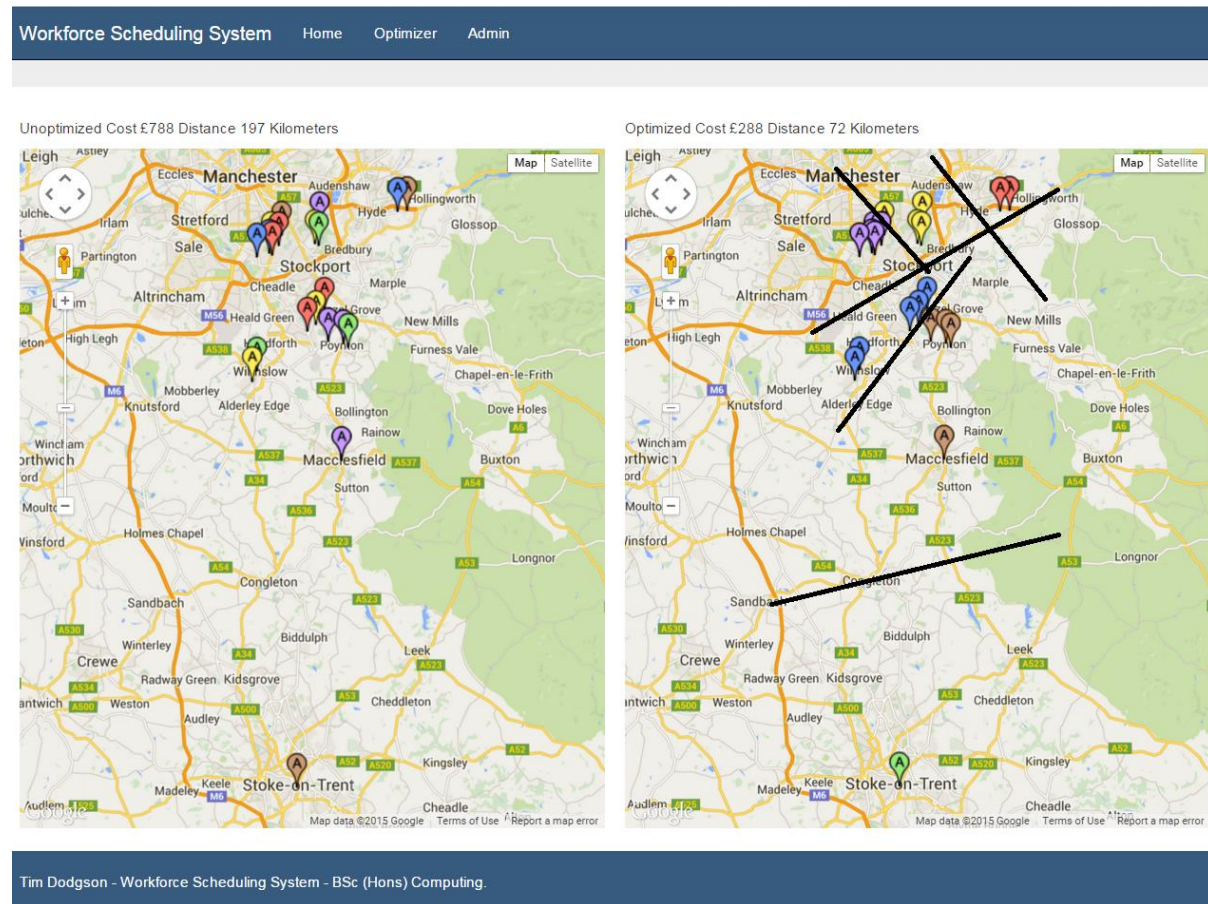


Figure 21 Patient clustering.

Chapter 6 Web Deployment and Configuration

6.1 Server Deployment

The web server will be deployed on a VPS (virtual private machine), the host will be <http://www.digitalocean.com> . Digital Ocean offer data centres across the world and a unique droplet system to store and backup your VPS along with good value for money.

The package offers 40 GB SSD (Solid State Drive) Disk, 2 Core Processor, 2 GB Memory and 3TB of Transfer at cost of £10 per calendar month. The system also only charges when the system is running.

Once the VPS is built, a username, password and IP (Internet Protocol) address is emailed to the user. Accessing the VPS from a Microsoft Windows computer can be achieved using Putty a free console application available from <http://www.putty.org/> .



Figure 22 Putty.

6.1.1 Operating system selection

Operating system selection is a very important question and basically comes down to Microsoft Windows or one of the flavours of Linux, most web servers are able to run on both operating systems with the exception of Internet Information Server. As ASP,.NET or one of the Microsoft databases are not used in this application Internet Information server will not be considered further. In General Microsoft Windows has a much larger footprint on system resources than Linux, as a virtual private server will be used to host the web server a Linux operating system is a much better choice due to minimal web server builds that have a very low footprint on system resources.

The chosen operating system is Ubuntu 14.04 LTS(Long Term Support) 32 bit.

6.1.2 Configuring Virtual Private server

Access to the server is via the console, once logged in as root (Linux Super User) the first step is to add a new user and configure the new user as a sudo.

```
adduser tim
usermod -a -G sudo tim
```

Next bring the VPS up to date, this is a two-step process first update the repository, next update the programs

```
sudo apt-get update
sudo apt-get upgrade
```


Next set time to UK and make permanent

```
sudo tzselect
sudo dpkg-reconfigure tzdata
```

6.1.3 Tomcat Web Server Installation and Configuration

Tomcat web server is used to serve Java applications to the internet using a technology called servlets.

Install Tomcat7

```
sudo apt-get install tomcat7
```

To test the installation navigate to <http://ipaddress:8080/>, the following web page should be displayed

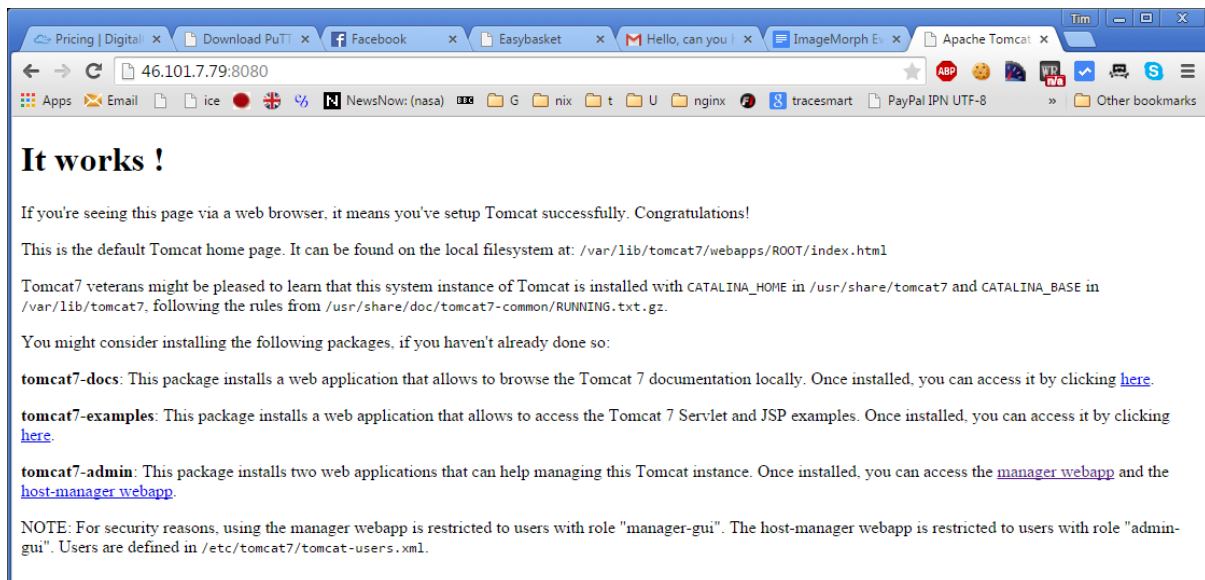


Figure 23 Tomcat it Works page.

The installation of Tomcat7 still needs additional packages to be installed to use the web administration panel

Install management webapps

```
sudo apt-get install tomcat7-docs tomcat7-admin tomcat7-examples
```

Install the JDK (Java Development Kit)

```
sudo apt-get install default-jdk
```

The next step is to configure the Tomcat Web Management Interface, this can be done by editing the tomcat-users.xml file.

```
sudo nano /etc/tomcat7/tomcat-users.xml
```

Delete the contents of the file and replace with text below this will add a user that has access to the manager-gui and admin-gui.

```
<tomcat-users>
  <user username="admin" password="password" roles="manager-gui,admin-gui"/>
</tomcat-users>
```

Save and quit the `tomcat-users.xml` file. For the changes to take effect a restart of the Tomcat server service is needed

```
sudo service tomcat7 restart
```

To access the web control panel navigate to <http://ipaddress:8080/manager/html>.

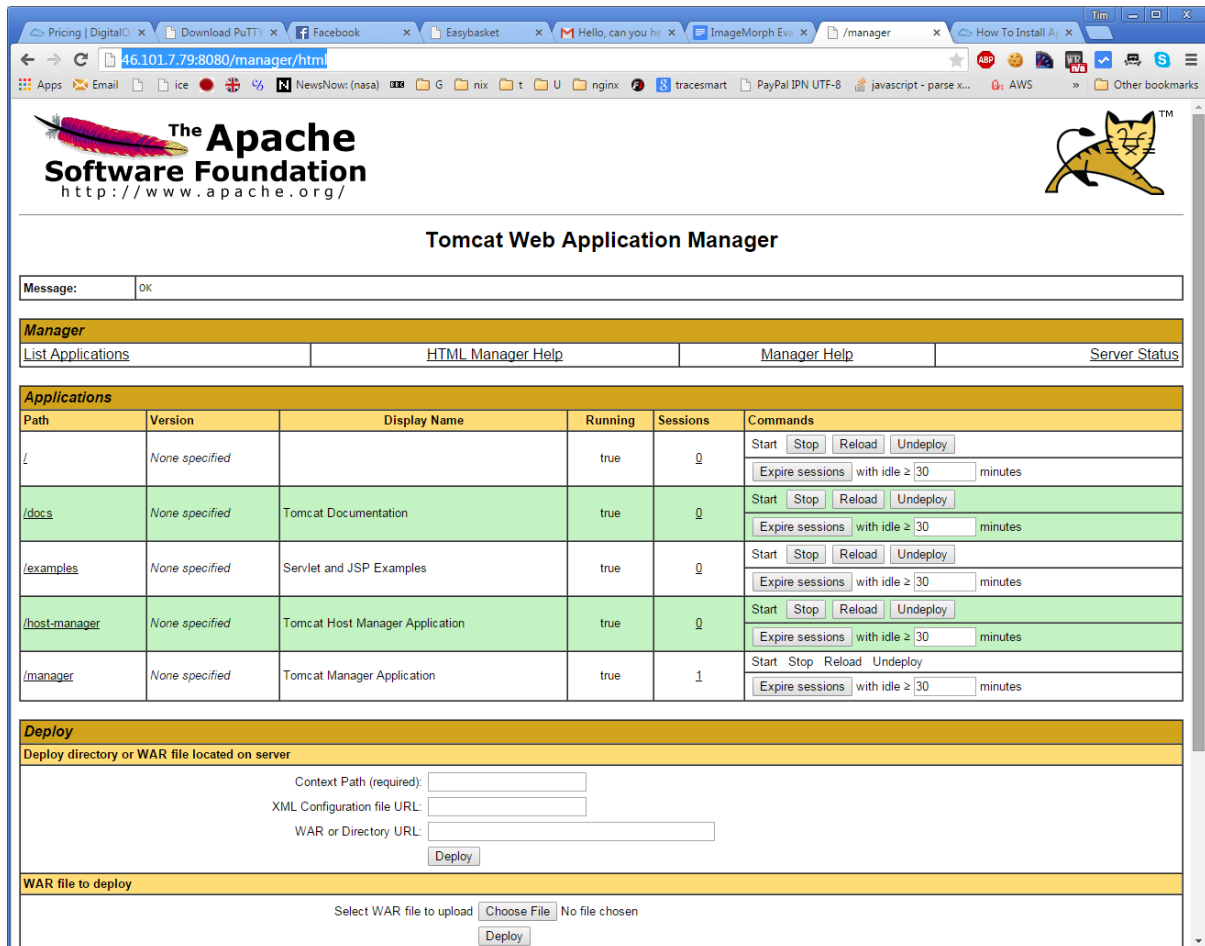


Figure 24 Tomcat it configuration page.

From this configuration page, web applications can be deployed, stopped, undeployed and restarted as well as access to help files and the server status page.

Some installation of Tomcat7 on a VPS can suffer from a lack of entropy resulting in Tomcat7 taking a long time to start.

Fix Slow start tomcat [5]

```
apt-get install haveged
```

6.1.3 Nginx

The reasons for using the Nginx server in front of the Tomcat web server are

1. To hide the real URL
2. Enables URL rewriting to produce human readable URLs

So for example the URL structure for this project below

`http://ipaddress:8080/Optimizer`

consists of IP address, then the 8080 Tomcat port number, the package name and finally the class name. The URL to access this with Nginx would be

`http://ipaddress/Optimizer/`

This is easier to remember from a human readable usability viewpoint and offers the following security benefits

1. Hides the port that Tomcat server is running
2. Hides the Java package name
3. Hides the servlet class name

Nginx also has a faster response time.

6.1.4 Install and configure

To install Nginx log into Linux console as a sudo and run the following commands

```
sudo apt-get update
sudo apt-get install nginx
```

Next open the following file to add a include directive

```
sudo nano /etc/nginx/nginx.conf
```

At the end of the 'http' section add the following

```
include /var/lib/tomcat7/*/nginx.conf;
```

Make a new file called `nginx.conf`, and save it in the root of your package and add the following to the file

```
server {
    server_name wfs.uk.to 46.101.7.79;
    root        /var/lib/tomcat7/webapps/ROOT/;

    location / {
        proxy_pass http://127.0.0.1:8080/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

Code 21: Nginx server block.

Save and close the file then restart Nginx

```
sudo service nginx restart
```

6.1.5 Deploying Workforce Scheduling Web Application

Deploying an application to a Tomcat server via the manager application. In eclipse right click the application and select export your application as a WAR (Web Archive) file.

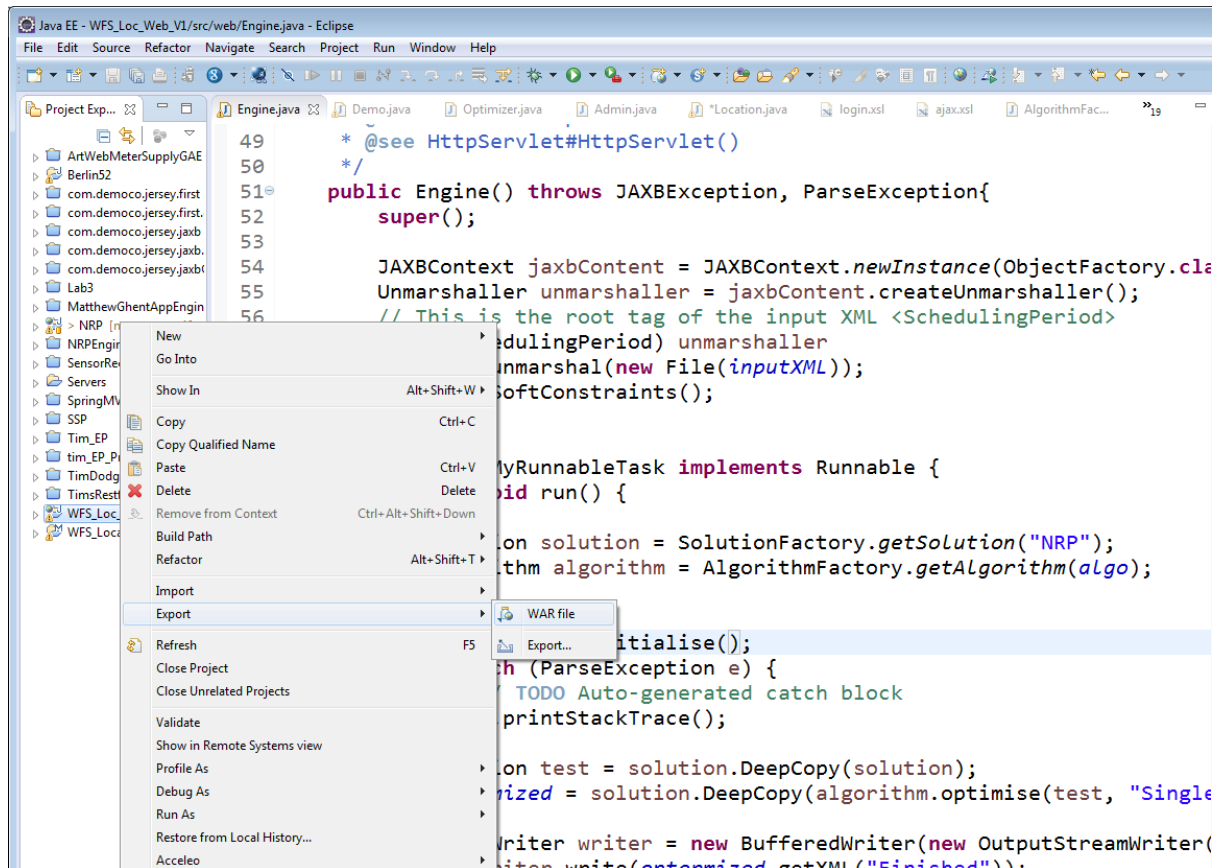


Figure 25 Create WAR file in eclipse.

The WAR file that is created contains your entire application including Java source code and your entire web root including images, css files, any static webpages and JavaScript.

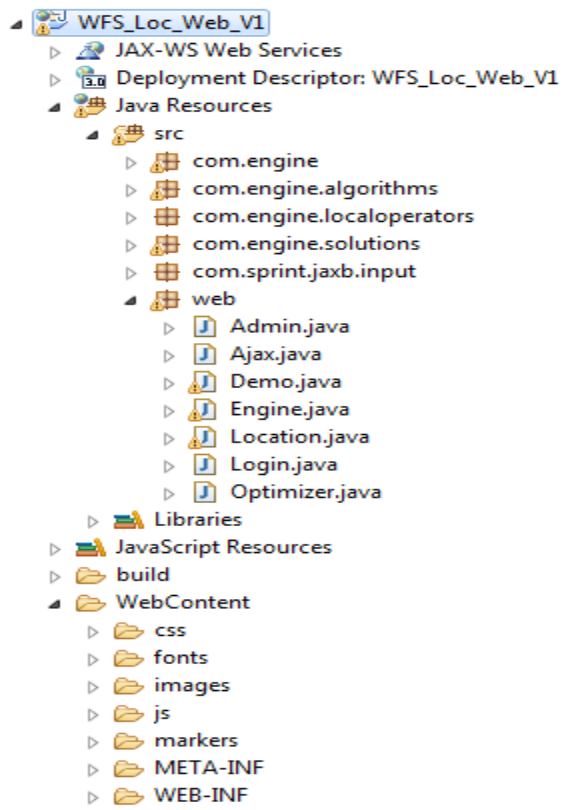


Figure 26 Contents of WAR file.

Next open the manager application at <http://ipaddress:8080/manager/html> and navigate to the choose file button, select the WAR file that was just created and click ok, lastly click deploy. The website can now be found at <http://ipaddress:8080/packageName/classname>

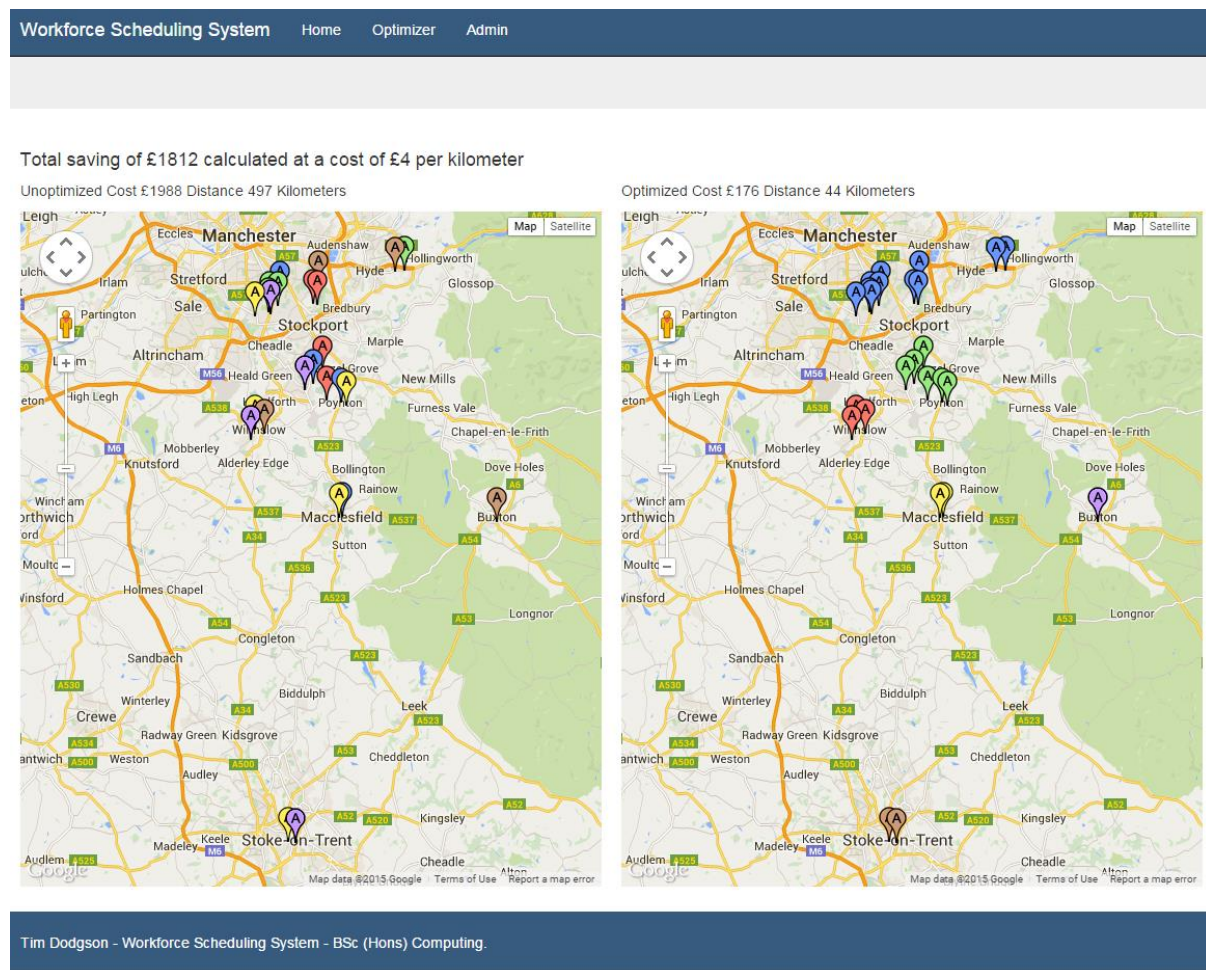


Figure 27 Deployed website.

6.2 Servlets

Tomcat server uses servlets to interact with http Get and Post requests a basic servlet class is shown below.

```
@WebServlet("/Demonstration")
public class Demonstration extends HttpServlet {
    private static final long serialVersionUID = 1L;

    public Demonstration() {
        super();
    }

    protected void doGet(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {

    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response) throws
ServletException, IOException {

    }
}
```

Code 22: Java servlet.

If a http Get request is sent to the server, the doGet() method in the servlet handles this request and for a http POST method would be handled by the doPost() method. Both these methods have two parameters a request and response, when a http request is sent to the Tomcat server the relevant method will handle it and on completion a response will be sent to the web page

The URL (Uniform Resource Lookup) to access this servlet is stored in the annotation at the begging of the class @WebServlet("/Demonstration") the full URL would be <http://ipaddress/pakagename/Demonstration>.

6.3 Transforming output

The optimiser will create a XML (EXtensible Markup Language) representation of the optimised schedule which can be passed to any application or transformed via XSL (EXtensible Stylesheet Language) Transform to be displayed in a browser as a web page. XSL Transforming gives you the ability to totally separate your web output from your business logic.

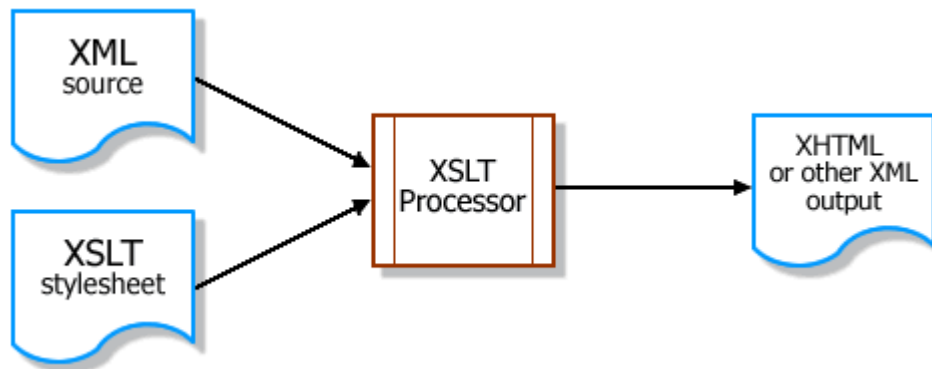


Figure 28 XSLT Processor.

The XSL processor requires a XML source document and a XSLT style sheet to generate the desired output. Code nn shoes the Java code to transform XML to HTML using XSL follows.

```
PrintWriter out = response.getWriter();
try {
    StringWriter writer = new StringWriter();
    TransformerFactory tFactory = TransformerFactory.newInstance();
    Transformer transformer =
        tFactory.newTransformer(new javax.xml.transform.stream.StreamSource("XSL_StyleSheet"));

    // Parameters passed to style sheet
    transformer.setParameter("from", "Demo");
    transformer.setParameter("user", user);
    transformer.transform(new javax.xml.transform.stream.StreamSource(new File("XML_Input")),
        new javax.xml.transform.stream.StreamResult(writer));

    String result = writer.toString();
    out.println("HTML_Output");
} catch (Exception e) {
    e.printStackTrace();
}
```

Code 23: Java code for XSL transform.

6.3.1 XSL

XSL is a language for creating a style sheet that describes how data stored in a XML document is to be presented. XSL gives a developer the tools to describe exactly which data fields in an XML document to display and exactly where and how to display them. Like any style sheet language, XSL can be used to create a style definition for one XML document or reused for many other XML documents. Code 23 demonstrates the use of XSL.

```
<!-- XML Document -->
<root>
  <text atr="xml-attribute">Hello World</text>
</root>

<!-- XSL Transform -->
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
  <xsl:output method="html" encoding="utf-8" indent="yes" />

  <xsl:template match="/">
    <xsl:text disable-output-escaping='yes'>&lt;!DOCTYPE html</xsl:text>
    <html>
      <head>
      </head>
      <body>
        <p>This is the text element attribute <xsl:value-of select="root/text/@atr"/></p>
        <p>This is the text element value <xsl:value-of select="root/text"/></p>
      </body>
    </html>
  </xsl:template>
</xsl:stylesheet>

<!-- HTML Output -->
<!DOCTYPE html>
<html>
  <head>
  </head>
  <body>
    <p>This is the text element attribute: xml-attribute </p>
    <p>This is the text element value: Hello World </p>
  </body>
</html>
```

Code 24: Demonstrates the use of XSL.

XSL is a functional programming language that support loops, conditional jumps, variables, recursion, and much more.

6.4 Bootstrap

Bootstrap is a front-end framework that can speed up development times of front-end website deployment. Bootstrap offers speed of development, ready-made design patterns and unifies naming conventions among teams. With the inclusion jQuery this makes for rapid consistent prototyping.

Bootstrap offers mobile first responsive web design, layout automatically adjusts depending on the media viewing the website. Fig 28 shows the difference between a browser on a desktop and a mobile phone.

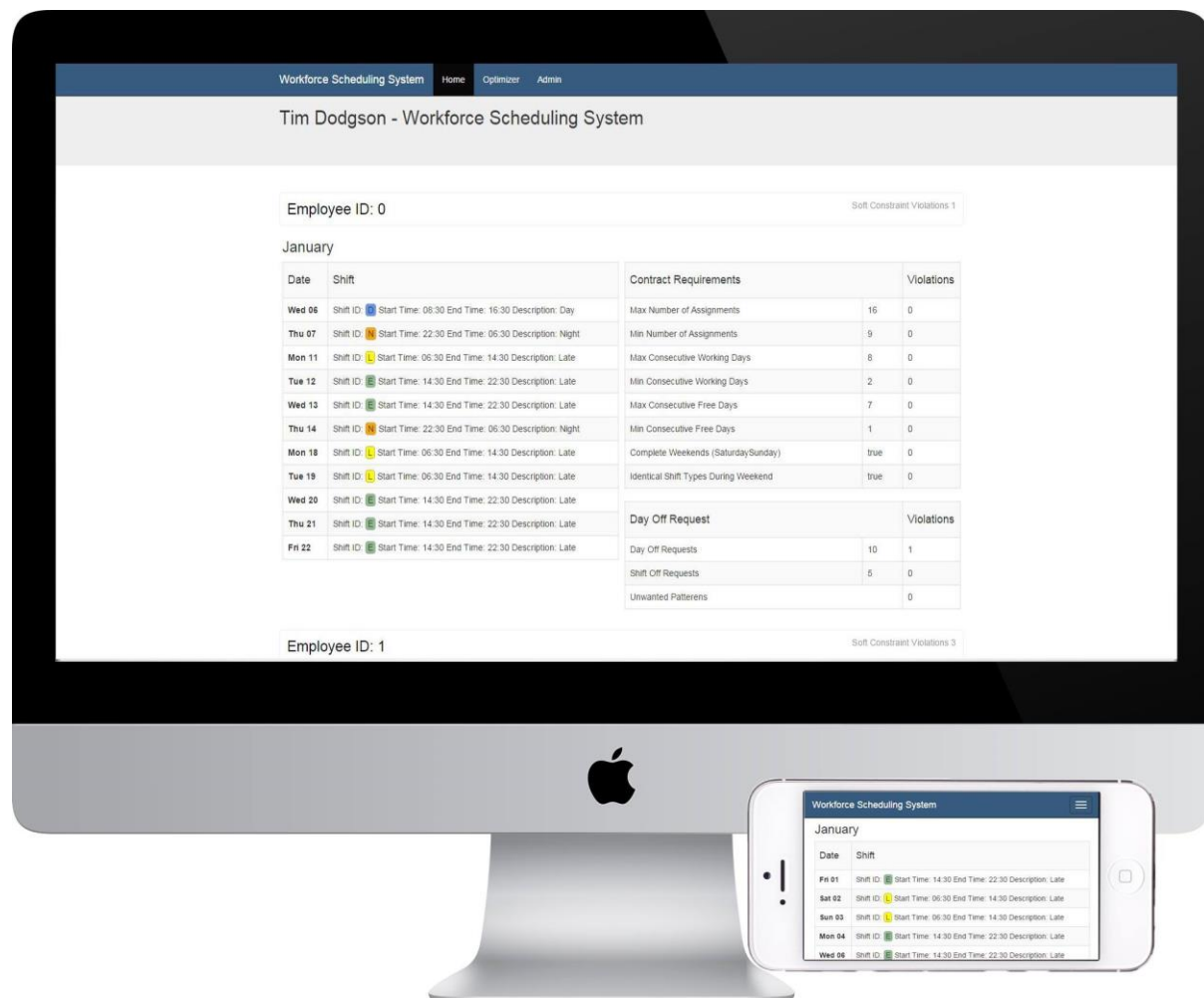


Figure 29 Responsive display.

Bootstrap as a standard set of css attributes for buttons, margins, padding, lineheight, forms and much more all conform to established standards for Desktop, Tablet and Mobile phone. Customising is as easy as adding a second css style sheet with your custom settings.

6.5 Understanding the schedule

Staff	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
Emp ID: 0						L	N				N	L	E	E	E	D	D	N		N	E	D	E	E				
Emp ID: 1	N	D	D	N				D	D	D			E	N						D	N	N			E	E	E	E
Emp ID: 2					L	N	D					D	L	L	E				D	D	E		E	N	N	E	D	D
Emp ID: 3	D	N	N	E	L		L	E			D	E						E	E			L	L	L			N	N
Emp ID: 4				E	D		E	N					L	E	D					L	L				L	E	L	
Emp ID: 5	E	L	L						L	L	L	E	E		L	N	N	L	E		D	E				L	L	L
Emp ID: 6				L	E	E							D	L								L	L			D	L	
Emp ID: 7	E			L	E	D		E	N	N	L				L	L	L			E	L		D	D		D	E	L
Emp ID: 8	L				E	L					L	L						E	N						L			
Emp ID: 9	L	E	E	D	N	L	E	L	E	E	E	N	N	D	N	E	E	L	L	L	E					N	N	E
Optimize Day	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA

Figure 30 Responsive display.

Figure 30 shows a roster for the period 1st January to 28th January through the use of colour a lot information can be gleaned by the operator figure 31 shows this in more detail.

Staff	01	02	03	04	05	06	07	08	09	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
Emp ID: 0						L	N				N	L	E	E	E	D	D	N		N	E	D	E	E				
Emp ID: 1	N	D	D	N				D	D	D			E	N						D	N	N			E	E	E	E
Emp ID: 2					L	N	D					D	L	L	E									N	E		D	D
Emp ID: 3	D	N	N	E	L		L	E			D	E											L	L			N	N
Emp ID: 4				E	D		E	N					L	E	D					L	L				L	E	L	
Emp ID: 5	E	L	L						L	L	L	E	E		L	N	N	L	E		D	E				L	L	L
Emp ID: 6				L	E	E							D	L								L	L			D	L	
Emp ID: 7	E			L	E	D		E	N	N	L				L	L	L			E	L		D	D		D	E	L
Emp ID: 8	L				E	L					L	L						E	N						L			
Emp ID: 9	L	E	E	D	N	L	E	L	E	E	E	N	N	D	N	E	E	L	L	L	E					N	N	E
Optimize Day	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA	AAA

Figure 31 Roster Explained.

Chapter 7 Conclusions and Future Work

7.1 Meta-Heuristics

Conclusions for meta-heuristics, after extensive testing of three trajectory based and one population based algorithms it has become quite clear that even though these algorithms are based on process in nature i.e. Simulated Annealing simulates the annealing process in metallurgy they all follow the same process;

- 1 Create a solution (solutions if population based) and find an initial value
- 2 Alter the solution slightly this is known as applying a local operator
- 2 Recalculate value
- 3 If equal or better save as new best value

It is at this point the algorithms differ, each algorithm has different mechanism to except a worse value in the hopes of jumping out of what is known as a local optima. As most algorithms delve deeper into the search space the chance they will except a worse value is lessened to the point of zero, this is the most optimized value this algorithm can achieve, with the exception Iterated Local Search which if given enough time to run this algorithm will hit the best optimization, this coupled with the fact there is one parameter to tune makes Iterated Local search a solid choice for this application and the results in chapter five bear this out.

7.2 Local Operators

Local operator are the degree by which you alter a solution, by having a number of different local operator that alter the solution by different amounts and randomly switching between them gives really good results on the kick phase of Iterated Local search.

7.3 Virtual Private Server

Virtual private sever selection is crucial to a reliable and stable web site, many deals are available from many supplies tests was carried out on five different providers before a final decision was made for the deployment.

7.4 Future work

In future work there are a considerable amount of direction to take for instance the raw power of infrastructure as a service offered by amazon web services, would give the ability to run much larger passes in the same amount of time as well as multi CPUs which have multiple cores offer the chance for parallelism which could take better advantage of a thread based programming in the optimizer. Algorithm design could be further improved in two ways for Iterated Local Search by firstly using the kick to control varying local operators which may give better coverage of the search space, another direction would be to save a copy(s) of the previous best Soultion(s) and add a equals method which would compare the new candidate to a previously saved, and only except if there differed. The result would be a narrowing of the search space.

Appendix

Terms of Reference

Tim Dodgson Terms of Reference Workforce Scheduling Web Application

1 Course Learning Outcomes

- To equip students with the practical skills and the theoretical underpinning necessary for the analysis, design and construction of information systems using a variety of methods.
- To enable students to gain an advanced understanding of database management systems, and to develop advanced skills in their exploitation.
- To develop the ability to design, specify, construct and maintain software.
- To equip students with skills and attributes that will enhance employment opportunities in general computing and IT.

2 Background

2.1 Workforce Scheduling

The general workforce scheduling problem involves the assignment of employees to specific working days and shifts in order to cover a demand. The problem size is determined by the planning horizon, number of employees and flexibility of the shifts. Further complexity is added when employees have to be assigned tasks within a shift.

A scheduling optimizer has to consider both hard constraints and soft constraints. Hard constraints are those which are always valid while soft constraints are those which we would like to be true. An assignment which satisfies all hard constraints is known as a feasible solution. The general goal is to schedule the resources to meet the hard constraints while aiming at a high quality result with respect to the soft constraints.

2.2 Nurse Rostering Problem (NRP)

Nurse rostering is a specific example of workforce scheduling where the overall goal is to assign shifts to nurses over a given planning horizon, while considering, laws and regulations. In nurse rostering, the hard constraint is that each nurse can only be assigned a single shift in a day, while soft constraints include day off requests and shift off requests. The output is a roster or timetable of working shifts for the nurses that also provides an overview of staff utilization and associated costs.

2.2.1 Goal of Nurse Rostering

The goal is to have nurse cover with the right skills 24 hours a day, 7 days a week, 365 days a year with as many soft constraints satisfied as possible, and have a schedule produced in a timely fashion.

2.2.2 Constraints

The main hard constraints for the nurse rostering problem is that each nurse can only be assigned a single

shift in a day e.g. early, day, late or night and all requested shifts are assigned.

Contract specific constraints are:-

- Patterns (i.e. to avoid working and early shift after a night shift)
- Maximum number of assignments during planning horizon
- Minimum number of assignments during planning horizon
- Minimum/maximum number of consecutive working days
- Minimum/maximum number of consecutive free working days
- Minimum/maximum consecutive working weekends
- Assign complete weekends
- Identical shift types during weekend
- No night shift before a free weekend

Employee constraints are:-

- Day off requests
- Day on requests
- Shifts off requests
- Shift on requests

2.2.3 NRP Competition

In 2010, a nurse rostering competition was organised and run by the CODES research group at Katholieke

Universiteit Leuven in Belgium, SINTEF Group in Norway and the University of Udine in Italy. A set of

data sets was made available and used as benchmark instances for testing and evaluating competition solutions. Competition dataset together with XML schemas can be downloaded from the competition web

page at

<https://www.kuleuven-kortrijk.be/nrpcompetition>.

An important aim of this competition is to generate new approaches to the associated problems by attracting

users from all areas of research. As with many cases in the past, significant advancements have been made in

research areas by attracting multi-disciplinary approaches.

2.3 Optimisation Algorithms for Scheduling

In general scheduling optimizers first solve hard constraints to achieve a feasible solution and then solve soft

constraints. Soft constraints can have a weight value assigned to ensure more important soft constraints are

utilized over less important ones. This will enable the system to better fulfil the needs of the workforce.

Scheduling optimizers attempt to satisfy as many soft constraints as possible to produce the best possible

solution in a given time frame. The cost function can be calculated as the number of soft constraint violations.

Algorithms used to solve optimisation and search problems can be subdivided into two categories, deterministic or stochastic. A deterministic algorithm performs the same way for a given set of initial conditions. In a stochastic algorithm randomness is present and so different runs produce different results.

Stochastic algorithms which incorporate randomisation and local search are defined as meta-heuristics.

Real-world optimization and searching problems such as workforce (employee) scheduling are NP hard, very

complex and time consuming. Two major approaches are traditionally used to tackle these problems: exact

methods and meta-heuristics. Exact methods find exact solutions by searching every solution combination

but are impractical as the search space or problem size increases. Conversely, meta-heuristics provide near

optimal solutions in a reasonable time.

Meta-heuristics can be subdivided into two further categories: trajectory-based meta-heuristics and population-based meta-heuristics. A trajectory-based technique starts with a single initial solution and, at

each step of the search, the current solution is replaced by another (often the best) solution found in its neighbourhood. Examples of trajectory based algorithms are hill climbing (and variants such as Late Acceptance Hill Climbing), tabu search and simulated annealing. Population-based algorithms make use of a

population of solutions. The initial population is normally randomly generated and then enhanced through an

iterative process. At each generation of the algorithm, the whole or part of the population is replaced by

newly generated (best) individuals. Examples of population based algorithms include genetic algorithms and

quantum annealing.

2.4 Web Applications and VPS (Virtual Private Server) Deployment

A virtual private server (VPS) is a virtual machine provided by an internet hosting company. A virtual private

server uses its own copy of an operating system (typically a Linux operating system is used), and users have

superuser access to that operating system instance. This means that users can install their own software that

is available for that OS. The physical server typically runs a hypervisor which is tasked with creating, releasing, and managing the resources of "guest" operating systems, or virtual machines. There are a couple

of different types of hypervisor. Type 1 hypervisors are installed before any guest machines and work directly with the underlying hardware. VMWare is an example of a type 1 hypervisor and is often used by

hosting companies. Type 2 hypervisors run on top of a traditional operating system with guests (virtual

machines) at another level above that. Virtualbox is an example of a type 2 hypervisor and can be installed

locally for prototyping a web application. However, as the number of virtualization clients that can typically

run on any single machine is limited, a VPS is supplied with fixed number of processor cores, RAM, and

disk space and these cannot be changed on-demand.

The focus of this project will be around solving the nurse rostering problem and deploying a nurse

scheduling web application. The project will study the performance of optimization algorithms for scheduling and their suitability for web deployment using a virtual private server.

4 Project aims

1. The main aim of the project is to develop and implement a Java framework for scheduling which will be applied to nurse rostering and deployed as a web application.
2. To manage project development and documentation.

4 Project Objectives

4.1 Core Objectives

Literature review objectives

- O1 Research existing employee (nurse) scheduling systems
- O2 Research the nurse rostering problem (NRP)
- O3 Study the constraints for nurse scheduling
- O4 Research NRP competition XML datasets and schemas
- O4 Compare Java frameworks for parsing XML (e.g. JAXB, DOM)
- O5 Research trajectory based meta-heuristics for scheduling (e.g. simulated annealing, hill climbing)
- O6 Research population based meta-heuristics for scheduling (e.g. genetic algorithms, quantum annealing)
- O7 Research web technologies (Servlets, JSP, Ajax, Server Side Events)
- O8 Research virtual servers for deploying a nurse scheduling system as a web application

Design objectives

- O10 Perform a requirements analysis
- O11 Undertake a UML design
- O12 Undertake user interface (UI) design

Implementation objectives

- O13 Develop and implement Java console based nurse scheduling system
- O14 Develop and implement trajectory based scheduling optimizers (SA, LAHC)
- O15 Manual tuning the parameters of optimisation algorithms
- O16 Deploy a nurse scheduling optimizer on a virtual private server
- O17 Add an administration page where constraints can be enabled and disabled and day/shift off/on requests, patterns, shifts, skills and employees can be added, edited or deleted
- O18 Add a configuration page where the settings, tuning parameters, and algorithm that will be used to produce the schedule can be adjusted
- O19 Add manual adjustment to the completed schedule where a user can fine tune the result by using drag & drop
- O20 Use version control for project backups

Testing objectives

- O21 Undertake unit testing, integration testing, system testing and usability testing
- O22 Undertake code optimization and undertake performance testing

Evaluation objectives

- O23 Perform computational experiments to compare the performance of different optimisation algorithms using NRP competition data sets
- O24 Analyse the performance and security of server deployment of the nurse scheduling system

4.2 Advanced Objectives

The advanced objective to apply to

AO1 Explore alternative optimization techniques such as iterative local search and multi-neighbourhood search

AO2 Auto tuning the parameters of optimisation algorithms

AO3 Implement a dispatcher for assigning tasks to shifts (e.g. using dispatching-rule based methods).

AO4 Explore methods for adding constraints dynamically.

5 Deliverables

The core deliverable will be a nurse scheduling system incorporating typical real world constraints using

trajectory based optimisation algorithms (SA, LAHC), which is deployed using a web server technology.

Advanced deliverables include;

- (i) alternative optimization algorithms.
- (ii) auto-tuning parameters of optimisation algorithms
- (iii) dispatcher for assigning tasks to shifts.
- (iv) dynamic constraints creation.

6) Required resources

- Oracle Java
- Eclipse Integrated Development Environment
- JAXB Java Architecture for XML Binding
- Apache Tomcat application server
- Nginx web server
- Open-source relational database management system (MySQL)
- Virtual Private Server (VPS)

7) Risk Analysis

Risk Likelihood Effect on project Mitigation strategy

Illness Medium High Apply for extension

Hardware failure

Low

High

Back up project code regularly using version control and online system

Unable to implement technical specification

Medium High Focus on core objectives

sprint_location_01.xml

```
<?xml version="1.0" encoding="utf-8" standalone="yes"?>
<SchedulingPeriod ID="sprint01" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:noNamespaceSchemaLocation="competition.xsd">
  <StartDate>2010-01-01</StartDate>
  <EndDate>2010-01-29</EndDate>
  <Skills>
    <Skill>Nurse</Skill>
  </Skills>
  <ShiftTypes>
    <Shift ID="E">
      <StartTime>06:30:00</StartTime>
      <EndTime>14:30:00</EndTime>
      <Description>Early</Description>
    </Shift>
  </ShiftTypes>
</SchedulingPeriod>
```

```

        <Hours>8</Hours>
        <Skills>
            <Skill>Nurse</Skill>
        </Skills>
    </Shift>
    <Shift ID="L">
        <StartTime>14:30:00</StartTime>
        <EndTime>22:30:00</EndTime>
        <Description>Late</Description>
        <Hours>8</Hours>
        <Skills>
            <Skill>Nurse</Skill>
        </Skills>
    </Shift>
    <Shift ID="D">
        <StartTime>08:30:00</StartTime>
        <EndTime>16:30:00</EndTime>
        <Description>Day shift</Description>
        <Hours>8</Hours>
        <Skills>
            <Skill>Nurse</Skill>
        </Skills>
    </Shift>
    <Shift ID="N">
        <StartTime>22:30:00</StartTime>
        <EndTime>06:30:00</EndTime>
        <Description>Night</Description>
        <Hours>8</Hours>
        <Skills>
            <Skill>Nurse</Skill>
        </Skills>
    </Shift>
</ShiftTypes>
<Patterns>
    <Pattern ID="0" weight="1">
        <PatternEntries>
            <PatternEntry index="0">
                <ShiftType>L</ShiftType>
                <Day>Any</Day>
            </PatternEntry>
            <PatternEntry index="1">
                <ShiftType>D</ShiftType>
                <Day>Any</Day>
            </PatternEntry>
        </PatternEntries>
    </Pattern>
    <Pattern ID="1" weight="1">
        <PatternEntries>
            <PatternEntry index="0">
                <ShiftType>D</ShiftType>
                <Day>Any</Day>
            </PatternEntry>
            <PatternEntry index="1">
                <ShiftType>E</ShiftType>
                <Day>Any</Day>
            </PatternEntry>
            <PatternEntry index="2">
                <ShiftType>D</ShiftType>
                <Day>Any</Day>
            </PatternEntry>
        </PatternEntries>
    </Pattern>
    <Pattern ID="2" weight="1">
        <PatternEntries>
            <PatternEntry index="0">
                <ShiftType>None</ShiftType>
                <Day>Friday</Day>
            </PatternEntry>
            <PatternEntry index="1">
                <ShiftType>Any</ShiftType>
                <Day>Saturday</Day>
            </PatternEntry>
            <PatternEntry index="2">
                <ShiftType>Any</ShiftType>

```

```

        <Day>Sunday</Day>
    </PatternEntry>
</PatternEntries>
</Pattern>
</Patterns>
<Contracts>
    <Contract ID="0">
        <SingleAssignmentPerDay weight="1">true</SingleAssignmentPerDay>
        <MaxNumAssignments weight="1" on="true">16</MaxNumAssignments>
        <MinNumAssignments weight="1" on="true">9</MinNumAssignments>
        <MaxConsecutiveWorkingDays weight="1" on="true">8</MaxConsecutiveWorkingDays>
        <MinConsecutiveWorkingDays weight="1" on="true">2</MinConsecutiveWorkingDays>
        <MaxConsecutiveFreeDays weight="1" on="true">7</MaxConsecutiveFreeDays>
        <MinConsecutiveFreeDays weight="1" on="true">1</MinConsecutiveFreeDays>
        <MaxConsecutiveWorkingWeekends weight="0" on="false">8</MaxConsecutiveWorkingWeekends>
        <MinConsecutiveWorkingWeekends weight="0" on="false">2</MinConsecutiveWorkingWeekends>
        <MaxWorkingWeekendsInFourWeeks weight="0" on="false">0</MaxWorkingWeekendsInFourWeeks>
        <WeekendDefinition>SaturdaySunday</WeekendDefinition>
        <CompleteWeekends weight="1">true</CompleteWeekends>
        <IdenticalShiftTypesDuringWeekend weight="1">true</IdenticalShiftTypesDuringWeekend>
        <NoNightShiftBeforeFreeWeekend weight="0">false</NoNightShiftBeforeFreeWeekend>
        <AlternativeSkillCategory weight="0">false</AlternativeSkillCategory>
        <UnwantedPatterns>
            <Pattern>0</Pattern>
            <Pattern>1</Pattern>
            <Pattern>2</Pattern>
        </UnwantedPatterns>
        <Description xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">fulltime</Description>
    </Contract>
    <Contract ID="1">
        <SingleAssignmentPerDay weight="1">true</SingleAssignmentPerDay>
        <MaxNumAssignments weight="1" on="true">12</MaxNumAssignments>
        <MinNumAssignments weight="1" on="true">6</MinNumAssignments>
        <MaxConsecutiveWorkingDays weight="1" on="true">5</MaxConsecutiveWorkingDays>
        <MinConsecutiveWorkingDays weight="1" on="true">2</MinConsecutiveWorkingDays>
        <MaxConsecutiveFreeDays weight="1" on="true">5</MaxConsecutiveFreeDays>
        <MinConsecutiveFreeDays weight="1" on="true">1</MinConsecutiveFreeDays>
        <MaxConsecutiveWorkingWeekends weight="0" on="false">5</MaxConsecutiveWorkingWeekends>
        <MinConsecutiveWorkingWeekends weight="0" on="false">2</MinConsecutiveWorkingWeekends>
        <MaxWorkingWeekendsInFourWeeks weight="0" on="false">0</MaxWorkingWeekendsInFourWeeks>
        <WeekendDefinition>SaturdaySunday</WeekendDefinition>
        <CompleteWeekends weight="1">true</CompleteWeekends>
        <IdenticalShiftTypesDuringWeekend weight="1">true</IdenticalShiftTypesDuringWeekend>
        <NoNightShiftBeforeFreeWeekend weight="0">false</NoNightShiftBeforeFreeWeekend>
        <AlternativeSkillCategory weight="0">false</AlternativeSkillCategory>
        <UnwantedPatterns>
            <Pattern>0</Pattern>
            <Pattern>1</Pattern>
            <Pattern>2</Pattern>
        </UnwantedPatterns>
        <Description xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">75_time</Description>
    </Contract>
    <Contract ID="2">
        <SingleAssignmentPerDay weight="1">true</SingleAssignmentPerDay>
        <MaxNumAssignments weight="1" on="true">9</MaxNumAssignments>
        <MinNumAssignments weight="1" on="true">4</MinNumAssignments>
        <MaxConsecutiveWorkingDays weight="1" on="true">4</MaxConsecutiveWorkingDays>
        <MinConsecutiveWorkingDays weight="1" on="true">2</MinConsecutiveWorkingDays>
        <MaxConsecutiveFreeDays weight="1" on="true">7</MaxConsecutiveFreeDays>
        <MinConsecutiveFreeDays weight="1" on="true">1</MinConsecutiveFreeDays>
        <MaxConsecutiveWorkingWeekends weight="0" on="false">4</MaxConsecutiveWorkingWeekends>
        <MinConsecutiveWorkingWeekends weight="0" on="false">2</MinConsecutiveWorkingWeekends>
        <MaxWorkingWeekendsInFourWeeks weight="0" on="false">0</MaxWorkingWeekendsInFourWeeks>
        <WeekendDefinition>SaturdaySunday</WeekendDefinition>
        <CompleteWeekends weight="1">true</CompleteWeekends>
        <IdenticalShiftTypesDuringWeekend weight="1">true</IdenticalShiftTypesDuringWeekend>
        <NoNightShiftBeforeFreeWeekend weight="0">false</NoNightShiftBeforeFreeWeekend>
        <AlternativeSkillCategory weight="0">false</AlternativeSkillCategory>
        <UnwantedPatterns>
            <Pattern>0</Pattern>
            <Pattern>1</Pattern>
            <Pattern>2</Pattern>
        </UnwantedPatterns>
    </Contract>

```



```

        <Description xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance">50_percent</Description>
    </Contract>
    <Contract ID="3">
        <SingleAssignmentPerDay weight="1">true</SingleAssignmentPerDay>
        <MaxNumAssignments weight="1" on="true">8</MaxNumAssignments>
        <MinNumAssignments weight="1" on="true">8</MinNumAssignments>
        <MaxConsecutiveWorkingDays weight="1" on="true">2</MaxConsecutiveWorkingDays>
        <MinConsecutiveWorkingDays weight="1" on="true">1</MinConsecutiveWorkingDays>
        <MaxConsecutiveFreeDays weight="1" on="true">20</MaxConsecutiveFreeDays>
        <MinConsecutiveFreeDays weight="1" on="true">3</MinConsecutiveFreeDays>
        <MaxConsecutiveWorkingWeekends weight="0" on="false">2</MaxConsecutiveWorkingWeekends>
        <MinConsecutiveWorkingWeekends weight="0" on="false">1</MinConsecutiveWorkingWeekends>
        <MaxWorkingWeekendsInFourWeeks weight="0" on="false">0</MaxWorkingWeekendsInFourWeeks>
        <WeekendDefinition>SaturdaySunday</WeekendDefinition>
        <CompleteWeekends weight="1">true</CompleteWeekends>
        <IdenticalShiftTypesDuringWeekend weight="1">true</IdenticalShiftTypesDuringWeekend>
        <NoNightShiftBeforeFreeWeekend weight="0">false</NoNightShiftBeforeFreeWeekend>
        <AlternativeSkillCategory weight="0">false</AlternativeSkillCategory>
        <UnwantedPatterns>
            <Pattern>0</Pattern>
            <Pattern>1</Pattern>
            <Pattern>2</Pattern>
        </UnwantedPatterns>
        <Description xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">night</Description>
    </Contract>
</Contracts>
<Employees>
    <Employee ID="0">
        <ContractID>0</ContractID>
        <Name>0</Name>
        <Cost>6</Cost>
        <Hours>8</Hours>
        <Locations>
            <Location lat="53.348654" lon="-2.121572" name="Poynton"/>
        </Locations>
        <Skills>
            <Skill>Nurse</Skill>
        </Skills>
    </Employee>
    <Employee ID="1">
        <ContractID>0</ContractID>
        <Name>1</Name>
        <Cost>7</Cost>
        <Hours>8</Hours>
        <Locations>
            <Location lat="53.367647" lon="-2.164072" name="Bramhall"/>
        </Locations>
        <Skills>
            <Skill>Nurse</Skill>
        </Skills>
    </Employee>
    <Employee ID="2">
        <ContractID>0</ContractID>
        <Name>2</Name>
        <Cost>8</Cost>
        <Hours>8</Hours>
        <Locations>
            <Location lat="53.442028" lon="-2.159697" name="Reddish"/>
        </Locations>
        <Skills>
            <Skill>Nurse</Skill>
        </Skills>
    </Employee>
    <Employee ID="3">
        <ContractID>0</ContractID>
        <Name>3</Name>
        <Cost>9</Cost>
        <Hours>8</Hours>
        <Locations>
            <Location lat="53.428160" lon="-2.202587" name="Burnage"/>
        </Locations>
        <Skills>
            <Skill>Nurse</Skill>

```

```

    </Skills>
  </Employee>
  <Employee ID="4">
    <ContractID>1</ContractID>
    <Name>4</Name>
    <Cost>5</Cost>
    <Hours>8</Hours>
    <Locations>
      <Location lat="53.328000" lon="-2.229409" name="Wilmslow"/>
    </Locations>
    <Skills>
      <Skill>Nurse</Skill>
    </Skills>
  </Employee>
  <Employee ID="5">
    <ContractID>1</ContractID>
    <Name>5</Name>
    <Cost>10</Cost>
    <Hours>8</Hours>
    <Locations>
      <Location lat="53.416600" lon="-2.231099" name="Didsbury"/>
    </Locations>
    <Skills>
      <Skill>Nurse</Skill>
    </Skills>
  </Employee>
  <Employee ID="6">
    <ContractID>2</ContractID>
    <Name>6</Name>
    <Cost>6</Cost>
    <Hours>11</Hours>
    <Locations>
      <Location lat="53.452977" lon="-2.082785" name="Hyde"/>
    </Locations>
    <Skills>
      <Skill>Nurse</Skill>
    </Skills>
  </Employee>
  <Employee ID="7">
    <ContractID>2</ContractID>
    <Name>7</Name>
    <Cost>12</Cost>
    <Hours>8</Hours>
    <Locations>
      <Location lat="53.002668" lon="-2.17940" name="Stoke-on-Trent"/>
    </Locations>
    <Skills>
      <Skill>Nurse</Skill>
    </Skills>
  </Employee>
  <Employee ID="8">
    <ContractID>3</ContractID>
    <Name>8</Name>
    <Cost>7</Cost>
    <Hours>8</Hours>
    <Locations>
      <Location lat="53.258663" lon="-2.119286" name="Macclesfield"/>
    </Locations>
    <Skills>
      <Skill>Nurse</Skill>
    </Skills>
  </Employee>
  <Employee ID="9">
    <ContractID>3</ContractID>
    <Name>9</Name>
    <Cost>7</Cost>
    <Hours>8</Hours>
    <Locations>
      <Location lat="53.259082" lon="-1.914829" name="Buxton"/>
    </Locations>
    <Skills>
      <Skill>Nurse</Skill>
    </Skills>
  </Employee>

```

```

</Employees>
<Patients>
  <Patient ID="0">
    <Days>
      <Day Time="Any" Day="ALL"/>
    </Days>
    <Name>0</Name>
    <HouseNumber>23</HouseNumber>
    <Street>Brookside Avenue</Street>
    <Area>Poynton</Area>
    <Postcode>SK121PW</Postcode>
    <Location lat="53.346134" lon="-2.114074"/>
  </Patient>
  <Patient ID="1">
    <Days>
      <Day Time="Any" Day="ALL"/>
    </Days>
    <Name>1</Name>
    <HouseNumber>1a</HouseNumber>
    <Street>Park Lane</Street>
    <Area>Poynton</Area>
    <Postcode>SK121RD</Postcode>
    <Location lat="53.348933" lon="-2.119714"/>
  </Patient>
  <Patient ID="2">
    <Days>
      <Day Time="Any" Day="ALL"/>
    </Days>
    <Name>2</Name>
    <HouseNumber>194</HouseNumber>
    <Street>Chester Road</Street>
    <Area>Poynton</Area>
    <Postcode>SK121HJ</Postcode>
    <Location lat="53.350277" lon="-2.139502"/>
  </Patient>
  <Patient ID="3">
    <Days>
      <Day Time="Any" Day="ALL"/>
    </Days>
    <Name>3</Name>
    <HouseNumber>26</HouseNumber>
    <Street>Newquay Drive</Street>
    <Area>Bramhall</Area>
    <Postcode>SK72HT</Postcode>
    <Location lat="53.363964" lon="-2.156828"/>
  </Patient>
  <Patient ID="4">
    <Days>
      <Day Time="Any" Day="ALL"/>
    </Days>
    <Name>4</Name>
    <HouseNumber>15</HouseNumber>
    <Street>Bickerton Drive</Street>
    <Area>Bramhall</Area>
    <Postcode>SK75QY</Postcode>
    <Location lat="53.374586" lon="-2.144858"/>
  </Patient>
  <Patient ID="5">
    <Days>
      <Day Time="Any" Day="ALL"/>
    </Days>
    <Name>5</Name>
    <HouseNumber>27</HouseNumber>
    <Street>Ack Lane East</Street>
    <Area>Bramhall</Area>
    <Postcode>SK72BE</Postcode>
    <Location lat="53.358664" lon="-2.167747"/>
  </Patient>
  <Patient ID="6">
    <Days>
      <Day Time="Any" Day="ALL"/>
    </Days>
    <Name>6</Name>
    <HouseNumber>22</HouseNumber>

```

```

        <Street>Felltop Drive</Street>
        <Area>Reddish</Area>
        <Postcode>SK56YS</Postcode>
    <Location lat="53.441255" lon="-2.150835"/>
</Patient>
<Patient ID="7">
    <Days>
        <Day Time="Any" Day="ALL"/>
    </Days>
    <Name>7</Name>
    <HouseNumber>14</HouseNumber>
    <Street>Lune Way</Street>
    <Area>Reddish</Area>
    <Postcode>SK57JH</Postcode>
    <Location lat="53.42556" lon="-2.15227"/>
</Patient>
<Patient ID="8">
    <Days>
        <Day Time="Any" Day="ALL"/>
    </Days>
    <Name>8</Name>
    <HouseNumber>5</HouseNumber>
    <Street>Fairford Way</Street>
    <Area>Reddish</Area>
    <Postcode>SK57NQ</Postcode>
    <Location lat="53.426794" lon="-2.157045"/>
</Patient>
<Patient ID="9">
    <Days>
        <Day Time="Any" Day="ALL"/>
    </Days>
    <Name>9</Name>
    <HouseNumber>26</HouseNumber>
    <Street>Glendale Avenue</Street>
    <Area>Burnage</Area>
    <Postcode>M191FF</Postcode>
    <Location lat="53.433594" lon="-2.201517"/>
</Patient>
<Patient ID="10">
    <Days>
        <Day Time="Any" Day="ALL"/>
    </Days>
    <Name>10</Name>
    <HouseNumber>70</HouseNumber>
    <Street>Overlea Drive</Street>
    <Area>Burnage</Area>
    <Postcode>M191LS</Postcode>
    <Location lat="53.425311" lon="-2.204518"/>
</Patient>
<Patient ID="11">
    <Days>
        <Day Time="Any" Day="ALL"/>
    </Days>
    <Name>11</Name>
    <HouseNumber>52</HouseNumber>
    <Street>Ashdale Drive</Street>
    <Area>Burnage</Area>
    <Postcode>M204TJ</Postcode>
    <Location lat="53.425437" lon="-2.214541"/>
</Patient>
<Patient ID="12">
    <Days>
        <Day Time="Any" Day="ALL"/>
    </Days>
    <Name>12</Name>
    <HouseNumber>4</HouseNumber>
    <Street>Grove Avenue</Street>
    <Area>Wilmslow</Area>
    <Postcode>SK95EG</Postcode>
    <Location lat="53.327631" lon="-2.234289"/>
</Patient>
<Patient ID="13">
    <Days>
        <Day Time="Any" Day="ALL"/>

```

```

    </Days>
    <Name>13</Name>
      <HouseNumber>9</HouseNumber>
      <Street>Knutsford Road</Street>
      <Area>Wilmslow</Area>
      <Postcode>SK96JA</Postcode>
      <Location lat="53.319727" lon="-2.240205"/>
</Patient>
<Patient ID="14">
  <Days>
    <Day Time="Any" Day="ALL"/>
  </Days>
  <Name>14</Name>
    <HouseNumber>51</HouseNumber>
    <Street>Whitechapel Street</Street>
    <Area>Didsbury</Area>
    <Postcode>M206TX</Postcode>
    <Location lat="53.416726" lon="-2.233966"/>
</Patient>
<Patient ID="15">
  <Days>
    <Day Time="Any" Day="ALL"/>
  </Days>
  <Name>15</Name>
    <HouseNumber>4</HouseNumber>
    <Street>Leyland Avenue</Street>
    <Area>Didsbury</Area>
    <Postcode>M206EW</Postcode>
    <Location lat="53.419797" lon="-2.21701"/>
</Patient>
<Patient ID="16">
  <Days>
    <Day Time="Any" Day="ALL"/>
  </Days>
  <Name>16</Name>
    <HouseNumber>589</HouseNumber>
    <Street>Kingsway</Street>
    <Area>Didsbury</Area>
    <Postcode>M191RA</Postcode>
    <Location lat="53.418399" lon="-2.21434"/>
</Patient>
<Patient ID="17">
  <Days>
    <Day Time="Any" Day="Saturday"/>
    <Day Time="Any" Day="Sunday"/>
  </Days>
  <Name>17</Name>
    <HouseNumber>10</HouseNumber>
    <Street>Paignton Avenue</Street>
    <Area>Hyde</Area>
    <Postcode>SK143HP</Postcode>
    <Location lat="53.451016" lon="-2.039307"/>
</Patient>
<Patient ID="18">
  <Days>
    <Day Time="Any" Day="ALL"/>
  </Days>
  <Name>18</Name>
    <HouseNumber>20</HouseNumber>
    <Street>Westwood Avenue</Street>
    <Area>Hyde</Area>
    <Postcode>SK143AZ</Postcode>
    <Location lat="53.452262" lon="-2.049578"/>
</Patient>
<Patient ID="19">
  <Days>
    <Day Time="Any" Day="ALL"/>
  </Days>
  <Name>19</Name>
    <HouseNumber>29</HouseNumber>
    <Street>Wardle Brook Avenue</Street>
    <Area>Hyde</Area>
    <Postcode>SK143JQ</Postcode>
    <Location lat="53.453084" lon="-2.036433"/>

```

```

</Patient>
<Patient ID="20">
  <Days>
    <Day Time="Any" Day="ALL"/>
  </Days>
  <Name>20</Name>
    <HouseNumber>64</HouseNumber>
    <Street>Selwyn Street</Street>
    <Area>Stoke-on-Trent</Area>
    <Postcode>ST41EE</Postcode>
    <Location lat="53.00029" lon="-2.181388"/>
</Patient>
<Patient ID="21">
  <Days>
    <Day Time="Any" Day="Monday"/>
    <Day Time="Any" Day="Wednesday"/>
    <Day Time="Any" Day="Friday"/>
    <Day Time="Any" Day="Saturday"/>
    <Day Time="Any" Day="Sunday"/>
  </Days>
  <Name>21</Name>
    <HouseNumber>12</HouseNumber>
    <Street>Penkhull New Road</Street>
    <Area>Stoke-on-Trent</Area>
    <Postcode>ST45DB</Postcode>
    <Location lat="53.000548" lon="-2.190031"/>
</Patient>
<Patient ID="22">
  <Days>
    <Day Time="Any" Day="Monday"/>
    <Day Time="Any" Day="Wednesday"/>
    <Day Time="Any" Day="Friday"/>
    <Day Time="Any" Day="Saturday"/>
    <Day Time="Any" Day="Sunday"/>
  </Days>
  <Name>22</Name>
    <HouseNumber>3</HouseNumber>
    <Street>Thorngrove Road</Street>
    <Area>Wilmslow</Area>
    <Postcode>SK91DD</Postcode>
    <Location lat="53.324282" lon="-2.222575"/>
</Patient>
<Patient ID="23">
  <Days>
    <Day Time="Any" Day="Wednesday"/>
    <Day Time="Any" Day="Saturday"/>
    <Day Time="Any" Day="Sunday"/>
  </Days>
  <Name>23</Name>
    <HouseNumber>5</HouseNumber>
    <Street>Rothwell Street</Street>
    <Area>Stoke-on-Trent</Area>
    <Postcode>ST45DL</Postcode>
    <Location lat="53.000638" lon="-2.195738"/>
</Patient>
<Patient ID="24">
  <Days>
    <Day Time="Any" Day="Monday"/>
    <Day Time="Any" Day="Wednesday"/>
    <Day Time="Any" Day="Friday"/>
    <Day Time="Any" Day="Saturday"/>
    <Day Time="Any" Day="Sunday"/>
  </Days>
  <Name>24</Name>
    <HouseNumber>8</HouseNumber>
    <Street>Green Street</Street>
    <Area>Macclesfield</Area>
    <Postcode>SK101JH</Postcode>
    <Location lat="53.258875" lon="-2.119567"/>
</Patient>
<Patient ID="25">
  <Days>
    <Day Time="Any" Day="Saturday"/>
    <Day Time="Any" Day="Sunday"/>
  </Days>

```

```

    </Days>
    <Name>25</Name>
    <HouseNumber>29</HouseNumber>
    <Street>Park Lane</Street>
    <Area>Macclesfield</Area>
    <Postcode>SK116TJ</Postcode>
    <Location lat="53.255733" lon="-2.125179"/>
  </Patient>
  <Patient ID="26">
    <Days>
      <Day Time="Any" Day="Monday"/>
      <Day Time="Any" Day="Tuesday"/>
      <Day Time="Any" Day="Thursday"/>
      <Day Time="Any" Day="Friday"/>
    </Days>
    <Name>26</Name>
    <HouseNumber>10</HouseNumber>
    <Street>Charlotte Street</Street>
    <Area>Macclesfield</Area>
    <Postcode>SK116JB</Postcode>
    <Location lat="53.25773" lon="-2.122906"/>
  </Patient>
  <Patient ID="27">
    <Days>
      <Day Time="Any" Day="Wednesday"/>
      <Day Time="Any" Day="Saturday"/>
      <Day Time="Any" Day="Sunday"/>
    </Days>
    <Name>27</Name>
    <HouseNumber>26</HouseNumber>
    <Street>Temple Road</Street>
    <Area>Buxton</Area>
    <Postcode>SK179BB</Postcode>
    <Location lat="53.250823" lon="-1.926027"/>
  </Patient>
  <Patient ID="28">
    <Days>
      <Day Time="Any" Day="Monday"/>
      <Day Time="Any" Day="Wednesday"/>
      <Day Time="Any" Day="Friday"/>
      <Day Time="Any" Day="Saturday"/>
      <Day Time="Any" Day="Sunday"/>
    </Days>
    <Name>28</Name>
    <HouseNumber>4</HouseNumber>
    <Street>Clough Street</Street>
    <Area>Buxton</Area>
    <Postcode>SK176LJ</Postcode>
    <Location lat="53.254204" lon="-1.913986"/>
  </Patient>
  <Patient ID="29">
    <Days>
      <Day Time="Any" Day="Saturday"/>
      <Day Time="Any" Day="Sunday"/>
    </Days>
    <Name>29</Name>
    <HouseNumber></HouseNumber>
    <Street>Monsal Avenue</Street>
    <Area>Buxton</Area>
    <Postcode>SK177TH</Postcode>
    <Location lat="53.255334" lon="-1.899189"/>
  </Patient>
</Patients>
<CoverRequirements>
  <DayOfWeekCover>
    <Day>Monday</Day>
    <Cover>
      <Shift>E</Shift>
      <Preferred>2</Preferred>
    </Cover>
    <Cover>
      <Shift>L</Shift>
      <Preferred>2</Preferred>
    </Cover>
  </DayOfWeekCover>

```

```

    <Cover>
      <Shift>D</Shift>
      <Preferred>1</Preferred>
    </Cover>
    <Cover>
      <Shift>N</Shift>
      <Preferred>1</Preferred>
    </Cover>
  </DayOfWeekCover>
</DayOfWeekCover>
<DayOfWeekCover>
  <Day>Tuesday</Day>
  <Cover>
    <Shift>E</Shift>
    <Preferred>2</Preferred>
  </Cover>
  <Cover>
    <Shift>L</Shift>
    <Preferred>2</Preferred>
  </Cover>
  <Cover>
    <Shift>D</Shift>
    <Preferred>1</Preferred>
  </Cover>
  <Cover>
    <Shift>N</Shift>
    <Preferred>1</Preferred>
  </Cover>
</DayOfWeekCover>
</DayOfWeekCover>
<DayOfWeekCover>
  <Day>Wednesday</Day>
  <Cover>
    <Shift>E</Shift>
    <Preferred>2</Preferred>
  </Cover>
  <Cover>
    <Shift>L</Shift>
    <Preferred>2</Preferred>
  </Cover>
  <Cover>
    <Shift>D</Shift>
    <Preferred>1</Preferred>
  </Cover>
  <Cover>
    <Shift>N</Shift>
    <Preferred>1</Preferred>
  </Cover>
</DayOfWeekCover>
</DayOfWeekCover>
<DayOfWeekCover>
  <Day>Thursday</Day>
  <Cover>
    <Shift>E</Shift>
    <Preferred>2</Preferred>
  </Cover>
  <Cover>
    <Shift>L</Shift>
    <Preferred>2</Preferred>
  </Cover>
  <Cover>
    <Shift>D</Shift>
    <Preferred>1</Preferred>
  </Cover>
  <Cover>
    <Shift>N</Shift>
    <Preferred>1</Preferred>
  </Cover>
</DayOfWeekCover>
</DayOfWeekCover>
<DayOfWeekCover>
  <Day>Friday</Day>
  <Cover>
    <Shift>E</Shift>
    <Preferred>2</Preferred>
  </Cover>
  <Cover>
    <Shift>L</Shift>

```



```

        <Preferred>2</Preferred>
    </Cover>
    <Cover>
        <Shift>D</Shift>
        <Preferred>1</Preferred>
    </Cover>
    <Cover>
        <Shift>N</Shift>
        <Preferred>1</Preferred>
    </Cover>
</DayOfWeekCover>
<DayOfWeekCover>
    <Day>Saturday</Day>
    <Cover>
        <Shift>E</Shift>
        <Preferred>1</Preferred>
    </Cover>
    <Cover>
        <Shift>L</Shift>
        <Preferred>1</Preferred>
    </Cover>
    <Cover>
        <Shift>D</Shift>
        <Preferred>1</Preferred>
    </Cover>
    <Cover>
        <Shift>N</Shift>
        <Preferred>1</Preferred>
    </Cover>
</DayOfWeekCover>
<DayOfWeekCover>
    <Day>Sunday</Day>
    <Cover>
        <Shift>E</Shift>
        <Preferred>1</Preferred>
    </Cover>
    <Cover>
        <Shift>L</Shift>
        <Preferred>1</Preferred>
    </Cover>
    <Cover>
        <Shift>D</Shift>
        <Preferred>1</Preferred>
    </Cover>
    <Cover>
        <Shift>N</Shift>
        <Preferred>1</Preferred>
    </Cover>
</DayOfWeekCover>
</CoverRequirements>
<DayOffRequests>
    <DayOff weight="1">
        <EmployeeID>0</EmployeeID>
        <Date>2010-01-02</Date>
    </DayOff>
    <DayOff weight="1">
        <EmployeeID>0</EmployeeID>
        <Date>2010-01-08</Date>
    </DayOff>
    <DayOff weight="1">
        <EmployeeID>0</EmployeeID>
        <Date>2010-01-07</Date>
    </DayOff>
    <DayOff weight="1">
        <EmployeeID>0</EmployeeID>
        <Date>2010-01-05</Date>
    </DayOff>
    <DayOff weight="1">
        <EmployeeID>0</EmployeeID>
        <Date>2010-01-15</Date>
    </DayOff>
    <DayOff weight="1">
        <EmployeeID>0</EmployeeID>
        <Date>2010-01-04</Date>
    </DayOff>

```

[illegible]

[illegible]

[illegible]

[illegible]


```

        <EmployeeID>9</EmployeeID>
        <Date>2010-01-16</Date>
    </DayOff>
    <DayOff weight="1">
        <EmployeeID>9</EmployeeID>
        <Date>2010-01-04</Date>
    </DayOff>
</DayOffRequests>
<ShiftOffRequests>
    <ShiftOff weight="1">
        <ShiftTypeID>E</ShiftTypeID>
        <EmployeeID>0</EmployeeID>
        <Date>2010-01-03</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>L</ShiftTypeID>
        <EmployeeID>0</EmployeeID>
        <Date>2010-01-13</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>N</ShiftTypeID>
        <EmployeeID>0</EmployeeID>
        <Date>2010-01-12</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>L</ShiftTypeID>
        <EmployeeID>0</EmployeeID>
        <Date>2010-01-04</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>D</ShiftTypeID>
        <EmployeeID>0</EmployeeID>
        <Date>2010-01-02</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>N</ShiftTypeID>
        <EmployeeID>1</EmployeeID>
        <Date>2010-01-18</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>N</ShiftTypeID>
        <EmployeeID>1</EmployeeID>
        <Date>2010-01-09</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>L</ShiftTypeID>
        <EmployeeID>1</EmployeeID>
        <Date>2010-01-27</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>L</ShiftTypeID>
        <EmployeeID>1</EmployeeID>
        <Date>2010-01-08</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>N</ShiftTypeID>
        <EmployeeID>1</EmployeeID>
        <Date>2010-01-19</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>L</ShiftTypeID>
        <EmployeeID>2</EmployeeID>
        <Date>2010-01-27</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>N</ShiftTypeID>
        <EmployeeID>2</EmployeeID>
        <Date>2010-01-09</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>L</ShiftTypeID>
        <EmployeeID>2</EmployeeID>
        <Date>2010-01-24</Date>
    </ShiftOff>

```


[illegible]

```

        <Date>2010-01-28</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>E</ShiftTypeID>
        <EmployeeID>8</EmployeeID>
        <Date>2010-01-23</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>E</ShiftTypeID>
        <EmployeeID>8</EmployeeID>
        <Date>2010-01-27</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>N</ShiftTypeID>
        <EmployeeID>9</EmployeeID>
        <Date>2010-01-01</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>L</ShiftTypeID>
        <EmployeeID>9</EmployeeID>
        <Date>2010-01-21</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>N</ShiftTypeID>
        <EmployeeID>9</EmployeeID>
        <Date>2010-01-10</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>N</ShiftTypeID>
        <EmployeeID>9</EmployeeID>
        <Date>2010-01-21</Date>
    </ShiftOff>
    <ShiftOff weight="1">
        <ShiftTypeID>L</ShiftTypeID>
        <EmployeeID>9</EmployeeID>
        <Date>2010-01-25</Date>
    </ShiftOff>
</ShiftOffRequests>
</SchedulingPeriod>

```

Sprint_location.xsd

```

<?xml version="1.0" encoding="utf-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    <xs:annotation>
        <xs:documentation>Schema for personnel scheduling problems.</xs:documentation>
    </xs:annotation>
    <xs:element name="SchedulingPeriod">
        <xs:complexType>
            <xs:sequence>
                <xs:element name="StartDate" type="xs:date" />
                <xs:element name="EndDate" type="xs:date" />
                <xs:element name="Schedule" type="Schedule" minOccurs="0" />
                <xs:element name="Skills" type="Skills" minOccurs="0" />
                <xs:element name="ShiftTypes" type="ShiftTypes" />
                <xs:element name="Patterns" type="Patterns" minOccurs="0" />
                <xs:element name="Contracts" type="Contracts" />
                <xs:element name="Employees" type="Employees" />
                <!-- Added for Location -->
                <xs:element name="Patients" type="Patients" />
                <!-- End -->
                <xs:element name="CoverRequirements" type="CoverRequirements" />
                <xs:element name="DayOffRequests" type="DayOffRequests" minOccurs="0" />
                <xs:element name="DayOnRequests" type="DayOnRequests" minOccurs="0" />
                <xs:element name="ShiftOffRequests" type="ShiftOffRequests" minOccurs="0" />
                <xs:element name="ShiftOnRequests" type="ShiftOnRequests" minOccurs="0" />
            </xs:sequence>
            <xs:attribute name="ID" type="xs:string" use="required" />
            <xs:attribute name="OrganisationID" type="xs:string" use="optional" />
        </xs:complexType>
    </xs:element>
    <xs:complexType name="Schedule">
        <xs:sequence>

```

```

        <xs:element name="Assignment" minOccurs="0" maxOccurs="unbounded">
            <xs:complexType>
                <xs:sequence>
                    <xs:element name="Date" type="xs:date"/>
                    <xs:element name="Employee" type="xs:string"/>
                    <xs:element name="ShiftType" type="xs:string"/>
                </xs:sequence>
            </xs:complexType>
        </xs:element>
    </xs:sequence>
</xs:complexType>
<xs:complexType name="Skills">
    <xs:sequence>
        <xs:element name="Skill" type="ID" minOccurs="0" maxOccurs="unbounded" />
    </xs:sequence>
</xs:complexType>
<xs:complexType name="ShiftTypes">
    <xs:sequence>
        <xs:element name="Shift" maxOccurs="unbounded">
            <xs:complexType>
                <xs:all>
                    <xs:element name="StartTime" type="xs:time" />
                    <xs:element name="EndTime" type="xs:time" />
                    <xs:element name="Description" type="xs:string" minOccurs="0" />
                    <xs:element name="Hours" type="xs:float" minOccurs="0" />
                    <xs:element name="Skills" minOccurs="0">
                        <xs:complexType>
                            <xs:sequence>
                                <xs:element name="Skill" type="ID" maxOccurs="unbounded" />
                            </xs:sequence>
                        </xs:complexType>
                    </xs:element>
                </xs:all>
                <xs:attribute name="ID" type="ID" use="required" />
            </xs:complexType>
        </xs:element>
    </xs:sequence>
</xs:complexType>
<xs:complexType name="Patterns">
    <xs:sequence maxOccurs="unbounded">
        <xs:element name="Pattern">
            <xs:complexType>
                <xs:all>
                    <xs:element name="PatternEntries">
                        <xs:complexType>
                            <xs:sequence minOccurs="2" maxOccurs="unbounded">
                                <xs:element name="PatternEntry">
                                    <xs:complexType>
                                        <xs:all>
                                            <xs:element name="ShiftType" type="xs:string" />
                                            <xs:element name="Day" type="WeekDayOrAny" />
                                        </xs:all>
                                        <xs:attribute name="index" />
                                    </xs:complexType>
                                </xs:element>
                            </xs:sequence>
                        </xs:complexType>
                    </xs:element>
                </xs:all>
                <xs:attribute name="weight" />
                <xs:attribute name="ID" type="xs:string" />
            </xs:complexType>
        </xs:element>
    </xs:sequence>
</xs:complexType>
<xs:complexType name="Contracts">
    <xs:sequence>
        <xs:element name="Contract" maxOccurs="unbounded">
            <xs:complexType>
                <xs:all>
                    <xs:element name="SingleAssignmentPerDay" type="WeightOnly" minOccurs="0" />
                    <xs:element name="MaxNumAssignments" type="OnAndWeight" minOccurs="0" />
                    <xs:element name="MinNumAssignments" type="OnAndWeight" minOccurs="0" />
                    <xs:element name="MaxConsecutiveWorkingDays" type="OnAndWeight" minOccurs="0" />
                </xs:all>
            </xs:complexType>
        </xs:element>
    </xs:sequence>
</xs:complexType>

```

```

<xs:element name="MinConsecutiveWorkingDays" type="OnAndWeight" minOccurs="0" />
<xs:element name="MaxConsecutiveFreeDays" type="OnAndWeight" minOccurs="0" />
<xs:element name="MinConsecutiveFreeDays" type="OnAndWeight" minOccurs="0" />
<xs:element name="MaxConsecutiveWorkingWeekends" type="OnAndWeight" minOccurs="0" />
<xs:element name="MinConsecutiveWorkingWeekends" type="OnAndWeight" minOccurs="0" />
<xs:element name="MaxWorkingWeekendsInFourWeeks" type="OnAndWeight" minOccurs="0" />
<xs:element name="WeekendDefinition" type="Weekend" minOccurs="0" />
<xs:element name="CompleteWeekends" type="WeightOnly" minOccurs="0" />
<xs:element name="IdenticalShiftTypesDuringWeekend" type="WeightOnly" minOccurs="0" />
<xs:element name="NoNightShiftBeforeFreeWeekend" type="WeightOnly" minOccurs="0" />
<xs:element name="TwoFreeDaysAfterNightShifts" type="WeightOnly" minOccurs="0" />
<xs:element name="AlternativeSkillCategory" type="WeightOnly" minOccurs="0" />
<xs:element name="UnwantedPatterns" minOccurs="0">
  <xs:complexType>
    <xs:sequence minOccurs="0" maxOccurs="unbounded">
      <xs:element name="Pattern" type="ID" />
    </xs:sequence>
  </xs:complexType>
</xs:element>
<xs:element name="Description" />
</xs:all>
<xs:attribute name="ID" type="ID" use="required" />
</xs:complexType>
</xs:element>
</xs:sequence>
</xs:complexType>
<xs:complexType name="Employees">
  <xs:sequence>
    <xs:element name="Employee" maxOccurs="unbounded">
      <xs:complexType>
        <xs:all>
          <xs:element name="ContractID" type="ID" />
          <xs:element name="Name" type="xs:string" minOccurs="0" />
          <xs:element name="Cost" type="xs:float" minOccurs="0" />
          <xs:element name="Hours" type="xs:float" minOccurs="0" />
          <!-- Added for employee start location -->
          <xs:element name="Locations">
            <xs:complexType>
              <xs:sequence>
                <xs:element name="Location">
                  <xs:complexType>
                    <xs:simpleContent>
                      <xs:extension base="xs:string">
                        <xs:attribute type="xs:float" name="lat"
use="required"/>
                        <xs:attribute type="xs:float" name="lon"
use="required"/>
                        <xs:attribute type="xs:string" name="name"
use="required"/>
                      </xs:extension>
                    </xs:simpleContent>
                  </xs:complexType>
                </xs:element>
              </xs:sequence>
            </xs:complexType>
          </xs:element>
          <!-- End -->
        </xs:all>
      </xs:complexType>
    </xs:element>
    <xs:element name="Skills" minOccurs="0">
      <xs:complexType>
        <xs:sequence>
          <xs:element name="Skill" type="xs:string" maxOccurs="unbounded" />
        </xs:sequence>
      </xs:complexType>
    </xs:element>
  </xs:all>
  <xs:attribute name="ID" type="ID" use="required" />
</xs:complexType>
</xs:element>
</xs:sequence>
</xs:complexType>
<!-- Added for location of Patients -->
<xs:complexType name="Patients">
  <xs:sequence>
    <xs:element name="Patient" maxOccurs="unbounded">

```

```

<xs:complexType>
  <xs:sequence>
    <xs:element name="Days">
      <xs:complexType>
        <xs:sequence>
          <xs:element name="Day" maxOccurs="unbounded">
            <xs:complexType>
              <xs:simpleContent>
                <xs:extension base="xs:string">
                  <xs:attribute type="Time" name="Time"
use="required"/>
                  <xs:attribute type="WeekDay" name="Day"
use="required"/>
                </xs:extension>
              </xs:simpleContent>
            </xs:complexType>
          </xs:element>
        </xs:sequence>
      </xs:complexType>
    </xs:element>
    <xs:element type="xs:string" name="Name"/>
    <xs:element type="xs:string" name="HouseNumber"/>
    <xs:element type="xs:string" name="Street"/>
    <xs:element type="xs:string" name="Area"/>
    <xs:element type="xs:string" name="Postcode"/>
    <xs:element name="Location">
      <xs:complexType>
        <xs:simpleContent>
          <xs:extension base="xs:string">
            <xs:attribute type="xs:float" name="Lat" use="required"/>
            <xs:attribute type="xs:float" name="Lon" use="required"/>
          </xs:extension>
        </xs:simpleContent>
      </xs:complexType>
    </xs:element>
  </xs:sequence>
  <xs:attribute type="ID" name="ID" use="required"/>
</xs:complexType>
</xs:element>
</xs:sequence>
</xs:complexType>
<!-- End -->
<xs:complexType name="ConstraintAttributes">
  <xs:attribute name="on" type="xs:boolean" use="optional" />
  <xs:attribute name="weight" type="xs:nonNegativeInteger" use="optional" />
</xs:complexType>
<xs:complexType name="OnAndWeight">
  <xs:simpleContent>
    <xs:extension base="xs:nonNegativeInteger">
      <xs:attribute name="on" type="xs:boolean" use="optional" />
      <xs:attribute name="weight" type="xs:nonNegativeInteger" use="optional" />
    </xs:extension>
  </xs:simpleContent>
</xs:complexType>
<xs:complexType name="WeightOnly">
  <xs:simpleContent>
    <xs:extension base="xs:boolean">
      <xs:attribute name="weight" type="xs:nonNegativeInteger" use="optional" />
    </xs:extension>
  </xs:simpleContent>
</xs:complexType>
<xs:complexType name="CoverRequirements">
  <xs:choice minOccurs="0" maxOccurs="unbounded">
    <xs:element name="DayOfWeekCover">
      <xs:complexType>
        <xs:sequence>
          <xs:element name="Day" type="WeekDay" />
          <xs:element name="Cover" type="Cover" maxOccurs="unbounded" />
        </xs:sequence>
      </xs:complexType>
    </xs:element>
    <xs:element name="DateSpecificCover">
      <xs:complexType>
        <xs:sequence>

```

```

        <xs:element name="Date" type="xs:date" />
        <xs:element name="Cover" type="Cover" maxOccurs="unbounded" />
    </xs:sequence>
</xs:complexType>
</xs:element>
</xs:choice>
</xs:complexType>
<xs:complexType name="Cover">
    <xs:sequence>
        <xs:choice>
            <xs:element name="Shift" type="ID" />
        </xs:choice>
        <xs:element name="Preferred" type="xs:nonNegativeInteger" minOccurs="0" />
    </xs:sequence>
</xs:complexType>
<xs:complexType name="DayOffRequests">
    <xs:sequence>
        <xs:element name="DayOff" minOccurs="0" maxOccurs="unbounded">
            <xs:complexType>
                <xs:sequence>
                    <xs:element name="EmployeeID" type="ID" />
                    <xs:element name="Date" type="xs:date" />
                </xs:sequence>
                <xs:attribute name="weight" type="xs:nonNegativeInteger" use="required" />
            </xs:complexType>
        </xs:element>
    </xs:sequence>
</xs:complexType>
<xs:complexType name="DayOnRequests">
    <xs:sequence>
        <xs:element name="DayOn" minOccurs="0" maxOccurs="unbounded">
            <xs:complexType>
                <xs:sequence>
                    <xs:element name="EmployeeID" type="ID" />
                    <xs:element name="Date" type="xs:date" />
                </xs:sequence>
                <xs:attribute name="weight" type="xs:nonNegativeInteger" use="required" />
            </xs:complexType>
        </xs:element>
    </xs:sequence>
</xs:complexType>
<xs:complexType name="ShiftOffRequests">
    <xs:sequence>
        <xs:element name="ShiftOff" minOccurs="0" maxOccurs="unbounded">
            <xs:complexType>
                <xs:sequence>
                    <xs:element name="ShiftTypeID" type="ID" />
                    <xs:element name="EmployeeID" type="ID" />
                    <xs:element name="Date" type="xs:date" />
                </xs:sequence>
                <xs:attribute name="weight" type="xs:nonNegativeInteger" use="required" />
            </xs:complexType>
        </xs:element>
    </xs:sequence>
</xs:complexType>
<xs:complexType name="ShiftOnRequests">
    <xs:sequence>
        <xs:element name="ShiftOn" minOccurs="0" maxOccurs="unbounded">
            <xs:complexType>
                <xs:sequence>
                    <xs:choice>
                        <xs:element name="ShiftTypeID" type="ID" />
                    </xs:choice>
                    <xs:element name="EmployeeID" type="ID" />
                    <xs:element name="Date" type="xs:date" />
                </xs:sequence>
                <xs:attribute name="weight" type="xs:nonNegativeInteger" use="required" />
            </xs:complexType>
        </xs:element>
    </xs:sequence>
</xs:complexType>
<xs:simpleType name="WeekDay">
    <xs:restriction base="xs:string">
        <xs:enumeration value="Sunday" />

```

```

        <xs:enumeration value="Monday" />
        <xs:enumeration value="Tuesday" />
        <xs:enumeration value="Wednesday" />
        <xs:enumeration value="Thursday" />
        <xs:enumeration value="Friday" />
        <xs:enumeration value="Saturday" />
        <!-- Added for location -->
        <xs:enumeration value="ALL" />
        <!-- End -->
    </xs:restriction>
</xs:simpleType>
<xs:simpleType name="WeekDayOrAny">
    <xs:restriction base="xs:string" />
</xs:simpleType>
<xs:simpleType name="Weekend">
    <xs:restriction base="xs:string">
        <xs:enumeration value="SaturdaySunday" />
        <xs:enumeration value="FridaySaturdaySunday" />
        <xs:enumeration value="FridaySaturdaySundayMonday" />
        <xs:enumeration value="SaturdaySundayMonday" />
    </xs:restriction>
</xs:simpleType>
<xs:simpleType name="ID">
    <xs:restriction base="xs:string">
        <xs:pattern value="([a-zA-Z0-9._])+" />
    </xs:restriction>
</xs:simpleType>
<!-- Added for location -->
<xs:simpleType name="Time">
    <xs:restriction base="xs:string">
        <xs:enumeration value="Any" />
        <xs:enumeration value="AM" />
        <xs:enumeration value="PM" />
    </xs:restriction>
</xs:simpleType>
<!-- End -->
</xs:schema>

```

References

W. S. Brownell and J. M. Lawer. (1976). Scheduling of work for required in continuous operations under alternative labour policies. *Mgmt Sci.* 22, 597-605

E. G. Keith. (1979) Operator scheduling. *AIEE Trans.* II, 37-41

Sachidanand V. Begur, David M. Miller, and Jerry R. Weaver. (1997) An integrated spatial dss for scheduling and routing home-health-care nurses. *Interfaces*, 27(4):35–48

B. Cheang, H. Li, A. Lim, and B. Rodrigues. (2003). Nurse rostering problems – a bibliographic survey. *European Journal of Operational Research*, 151(3):447–460, December

V. Cerny. (1985) A thermodynamical approach to the travelling salesman problem: An efficient simulation algorithm. *Journal of Optimization Theory and Applications*, 45:41–51

S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. (1983) Optimization by simulated annealing. *Science*, 13 May 1983, 220(4598):671–680

H. R. Lourenc_o, O. Martin, and T. Stutzle. (2001) A beginner’s introduction to Iterated Local Search. In *Proceedings of MIC’2001 – Meta-heuristics International Conference*, volume 1, pages 1–6, Porto–Portugal

H. R. Lourenc_o, O. Martin, and T. Stutzle. (2002) Iterated local search. In F. Glover and G. Kochenberger, editors, Handbook of Metaheuristics, volume 57 of International Series in Operations Research & Management Science, pages 321–353. Kluwer Academic Publishers, Norwell, MA

O. Martin, S. W. Otto, and E. W. Felten. (1991) Large-step markov chains for the traveling salesman problem. Complex Systems, 5(3):299–326

T. Stutzle. (1999) Iterated local search for the quadratic assignment problem. Technical report aida-99-03, FG Intellektik, TU Darmstadt

T. Stutzle. (1999) Local Search Algorithms for Combinatorial Problems - Analysis, Algorithms and New Applications. DISKI - Dissertationen zur Kunstlik " en Intelligenz. infix, Sankt Augustin, Germany

Nurse Rostering Competition. (2010) Nurse rostering benchmark datasets [online] [Accessed on 10th November 2014], Available from < <http://allserv.kahosl.be/~pieter/nurserostering.html>>

Wikipedia(Hill Climbing). (2014) Hill climbing [online] [Accessed on 12th November 2014], Available from < http://en.wikipedia.org/wiki/Hill_climbing>

C S Nott. (2014) Late Acceptance Hill-Climbing algorithm (LAHC) [online] [Accessed on 11th November 2014], Available from <<http://www.cs.nott.ac.uk/~yxb/LAHC/>>

StackOverflow. (2014) JAXB vs DOM and SAX [online] [Accessed on 19th November 2014], Available from <<http://stackoverflow.com/questions/7709928/jaxb-vs-dom-and-sax>>

Peter Lin. (2009) So You Want High Performance [online] [Accessed on 17th November 2014], Available from < <https://tomcat.apache.org/articles/performance.pdf>>

Wikipedia (Java Servlet). (2014) Java Servlet [online] [Accessed on 16th November 2014], Available from < <https://tomcat.apache.org/articles/performance.pdf>>

Wikipedia (XML). (2014) XML [online] [Accessed on 03th November 2014], Available from < <http://en.wikipedia.org/wiki/XML>>

Wikipedia (XSL). (2014) XSL [online] [Accessed on 04th November 2014], Available from < <http://en.wikipedia.org/wiki/XSL>>

[58] Twitter Bootstrap. (2014) Bootstrap - The world's most popular mobile-first and responsive front-end framework [online] [Accessed on 07th November 2014], Available from < <http://getbootstrap.com/>>

[1] <http://infolab.stanford.edu/lore/home/index.html>

[2] <http://violin.qwer.tk/dusty/thesis/html/node6.html>

[3] <http://www.w3.org/>

[4] <https://docs.oracle.com/cd/E19316-01/819-3669/bnazf/index.html>

[5] <http://stackoverflow.com/questions/26431922/tomcat7-starts-too-late-on-ubuntu-14-04-x64-digitalocean>